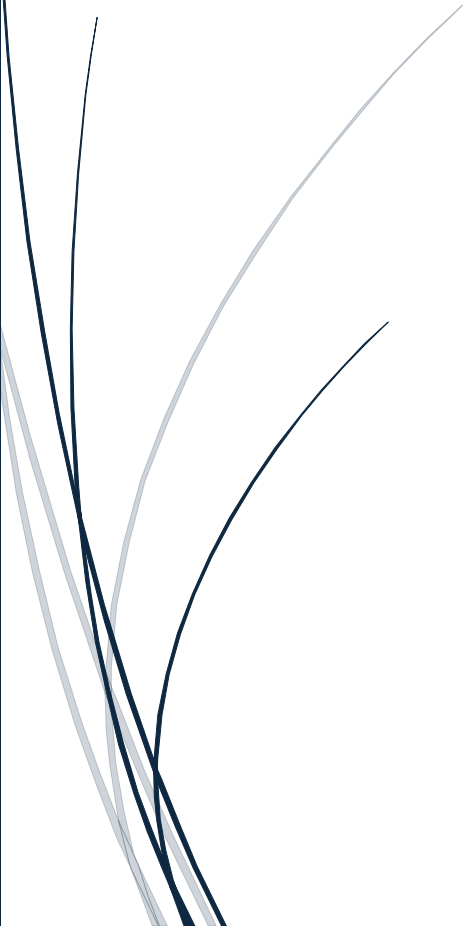




29/01/2025

NewMonopoly

Progetto per Ingegneria del Software



903321 Messa Alessandro
899659 Ronchi Matteo
899835 Tentori Francesca
894437 Teruzzi Luca
DEBANCHERZ

Sommario

Guida utente.....	3
Come utilizzare il gioco.....	3
Regolamento del gioco	3
1. Creazione della partita	3
2. Accesso alla partita.....	3
3. Giocatori aggiuntivi durante la partita.....	3
4. Svolgimento del gioco	3
5. La prigionie	4
6. Punti fedeltà	4
7. Abbandona partita e bancarotta	4
8. Fine della partita	4
1 Capitolo: REQUISITI	5
1.1 Introduzione.....	5
1.2 Caratteristiche principali	5
1.3 Requisiti funzionali	5
1.4 Requisiti non funzionali.....	6
1.5 Casi d'Uso	7
1.6 Diagramma dei Casi d'Uso	8
2 Capitolo: ANALISI.....	9
2.1 Introduzione.....	9
2.2 Modello di dominio	9
2.3 Diagramma di sequenza di sistema.....	10
2.3.1 Caso d'Uso: CreaPartita	10
2.3.2 Caso d'Uso: ConnessionePartita.....	11
2.3.3 Caso d'Uso: AbbandonaPartita	11
2.3.4 Caso d'Uso: AcquistaProprietà	12
2.3.5 Caso d'Uso: LanciaDadi	12
2.3.6 Caso d'Uso: PassaDalVia.....	12
2.3.7 Caso d'Uso: EffettuaTurno	13
2.3.8 Caso d'Uso: ScambiaProprietà	14
2.3.9 Caso d'Uso: ScegliPedina	14
2.3.10 Caso d'Uso: SpostaPedina.....	15
2.3.11 Caso d'Uso: LasciaPrigione.....	16
2.3.12 Caso d'Uso: GestisciProprietà	17
3 Capitolo: PROGETTAZIONE	18

3.1	Introduzione	18
3.2	Diagramma dell'architettura software	19
3.2.1	Frontend	20
3.2.2	EventBus	20
3.2.3	Backend	21
3.3	Diagramma delle classi a livello di progettazione	25
3.4	Diagramma di interazione: diagrammi di sequenza	26
3.4.1	Connessione partita	26
3.4.2	Crea partita	26
3.5	Diagramma a stati	27
3.5.1	Proprietà	27
3.5.2	Partita	27
3.5.3	Giocatore	27
3.6	Diagramma delle attività	28
3.6.1	Crea Partita	28
3.6.2	Connessione Partita	29
3.6.3	Abbandona Partita	30
3.7	Diagramma UML del database	31
4	Capitolo: IMPLEMENTAZIONE	33
4.1	Introduzione	33
4.2	Pattern e Principi SOLID	33
4.2.1	Pattern Architetturali	33
4.2.2	Design Pattern:	33
4.2.3	Principi Solid:	34
4.3	Refactoring	34
4.4	Test e qualità del codice	35

GUIDA UTENTE

COME UTILIZZARE IL GIOCO

Attualmente, il gioco è disponibile esclusivamente in modalità locale. Per avviare il gioco è necessario aprire il progetto del backend utilizzando un ambiente di sviluppo integrato (IDE) come ad esempio IntelliJ. Successivamente, eseguire l'applicazione dalla classe **NewMonopolyApplication**.

Una volta avviata l'applicazione, è possibile accedere al gioco aprendo un browser e digitando l'indirizzo: <http://localhost:8080/>.

REGOLAMENTO DEL GIOCO

1. Creazione della partita

Un giocatore assume il ruolo di amministratore (admin) per creare la partita. L'admin inserisce il suo nome utente e sceglie:

- **Livello di difficoltà:** Facile, Medio o Difficile. In base al livello scelto, i prezzi per acquistare le proprietà e le case aumenteranno rispettivamente del 5% (Medio) e del 10% (Difficile).
- **Randomizzazione:** se attivata, le proprietà saranno disposte in ordine casuale e avranno dei prezzi differenti.

Al termine della configurazione, viene generato un codice univoco che permette agli altri utenti di accedere alla partita.

2. Accesso alla partita

Gli altri partecipanti accedono alla partita già esistente inserendo il proprio nome (univoco rispetto agli altri giocatori) e il codice univoco della partita.

Dopo l'accesso, ciascun giocatore sceglie una pedina tra quelle disponibili. Le pedine scelte vengono aggiornate in tempo reale per evitare duplicati.

Una volta che tutti i giocatori sono entrati, l'admin potrà avviare il gioco. L'ordine di gioco sarà stabilito in base all'ordine di ingresso nella partita. Ogni partecipante inizierà con un saldo di 1500€.

Durante la partita c'è la possibilità di comunicare con gli altri giocatori attraverso la game chat, o vedere le azioni degli altri attraverso la game console.

3. Giocatori aggiuntivi durante la partita

Un giocatore che si unisce a una partita già iniziata, detto imprenditore, subirà alcuni svantaggi:

- Il suo saldo iniziale sarà ridotto a 1000€;
- I suoi punti fedeltà verranno svalutati.

4. Svolgimento del gioco

Durante il proprio turno, ogni giocatore lancia i dadi per determinare il numero di caselle da percorrere sul tabellone. In base alla casella in cui si fermano, potrà:

- Acquistare la casella, se non è già stata acquistata da un altro giocatore;
- Pagare l'affitto, se la casella appartiene a un altro giocatore;
- Seguire le istruzioni speciali, per caselle come Probabilità, Imprevisti o Tassa.

Se un giocatore lancia un dado doppio ha il diritto di rilanciarlo, ma non oltre tre volte consecutive. Al terzo doppio consecutivo, finirà in prigione.

I giocatori possono effettuare scambi di proprietà durante il proprio turno. Inoltre, se possiedono tutte le proprietà dello stesso colore, hanno la possibilità di costruire case e hotel sulle loro proprietà per incrementare gli affitti.

Durante lo scambio bisogna selezionare la propria proprietà che vogliamo scambiare e successivamente selezionare la proprietà che vogliamo. C'è l'opportunità anche di richiedere o offrire ulteriori soldi. In particolare, se si mettono i soldi in positivo io offro la mia proprietà e dei soldi, se invece li metto negativi io offro la mia proprietà in cambio di un'altra proprietà e dei soldi.

5. La prigione

Un giocatore finisce in prigione se:

- Atterra sulla casella "Vai in prigione";
- Lancia tre volte consecutive dadi doppi;
- Pesca una carta speciale che lo manda in prigione.

Una volta in prigione, il giocatore deve attendere tre turni a meno che non:

- Pesci una carta "Esci gratis di prigione".
- Paghi per uscire di prigione.
- Ottenga un doppio lanciando i dadi.

6. Punti fedeltà

I giocatori guadagnano punti fedeltà ogni volta che ricevono un affitto. Questi punti possono essere utilizzati per acquistare proprietà o pagare affitti. Il valore dei punti varia in base al livello di difficoltà:

- Facile: 1€ equivale a 2 punti (4 punti per l'imprenditore);
- Medio: 1€ equivale a 4 punti (8 punti per l'imprenditore);
- Difficile: 1€ equivale a 6 punti (16 punti per l'imprenditore);

7. Abbandona partita e bancarotta

Se un giocatore abbandona, le sue proprietà andranno alla banca. Se, invece, tutti i giocatori abbandonano, la partita termina.

Nel caso in cui un giocatore abbia un saldo negativo durante il proprio turno, può ipotecare le sue proprietà per saldare il debito. Se termina il turno ancora in debito, viene eliminato dalla partita.

8. Fine della partita

La partita prosegue fino a quando tutti i giocatori, eccetto uno, vanno in bancarotta o abbandonano la partita. Il vincitore sarà l'unico giocatore rimasto con denaro e proprietà.

1 CAPITOLO: REQUISITI

1.1 INTRODUZIONE

NewMonopoly è una versione avanzata e configurabile del classico gioco da tavolo Monopoly, progettata per essere giocata online in tempo reale su una piattaforma client-server.

I giocatori, che potranno essere da 2 a 6, possono accedere al gioco creando o partecipando a partite online. Una volta entrati i giocatori si muovono intorno al tabellone acquistando, scambiando o vendendo proprietà, costruendo case e hotel, pescando le carte Probabilità e Imprevisti, e raccogliendo affitti dai loro avversari. L'obiettivo è mandare in bancarotta gli avversari.

A differenza del classico Monopoly, sarà possibile avere dei diversi livelli di difficoltà, avere una randomizzazione delle caselle e dei prezzi e inoltre permetterà agli utenti che si vogliono unire in una partita già iniziata di poterlo fare, con però degli svantaggi.

1.2 CARATTERISTICHE PRINCIPALI

Con NewMonopoly ogni partita diventa un'esperienza unica e personalizzata. Il tabellone mantiene la classica struttura a 40 tabelle, ma colui che crea la partita avrà la possibilità di attivare una sequenza casuale di caselle e dei prezzi delle proprietà, rendendo ogni partita originale e imprevedibile.

NewMonopoly inoltre offre tre livelli di complessità:

- Facile: prezzi standard delle proprietà e vantaggi maggiori per i punti fedeltà;
- Medio: prezzi leggermente aumentati (+5%) e un bilanciamento più rigoroso nei vantaggi;
- Difficile: prezzi aumentati (+10%) e vincoli economici stringenti.

Una delle innovazioni più interessanti è il programma fedeltà. I giocatori accumulano punti fedeltà riscuotendo affitti. Questi punti possono essere utilizzati o combinati con i soldi per pagare affitti o comprare proprietà.

NewMonopoly consente a nuovi giocatori di unirsi a una partita in corso come "imprenditore". Questi giocatori iniziano con un saldo ridotto e una gestione differente dei punti fedeltà.

Il gioco è progettato per partite online con un massimo di sei e un minimo di due partecipanti. Il creatore della partita (admin) invierà agli altri giocatori un codice univoco per accedere alla partita. Ogni giocatore, come nel classico Monopoly, è rappresentato da una pedina personalizzata e inizia con un budget iniziale predefinito.

1.3 REQUISITI FUNZIONALI

Gestione dei giocatori:

- Supporto per 2-6 giocatori;
- Assegnazione di pedine ai giocatori;
- Assegnazione del bilancio iniziale;
- Se un giocatore abbandona la partita, le sue proprietà verranno date alla banca;
- Se tutti i giocatori abbandonano, la partita termina;
- Permettere l'inserimento di nuovi giocatori come imprenditori con un saldo iniziale ridotto e una gestione differente dei punti fedeltà.

Gestione del tabellone:

- Implementazione delle 40 caselle con i dettagli relativi;
- Possibilità di abilitare una sequenza casuale delle caselle (opzione configurabile).

Gestione meccanica di gioco:

- Movimento delle pedine determinato dal lancio dei dadi;
- Acquisto e scambio di proprietà tra giocatori;
- Costruzione di case e hotel sulle proprietà possedute;
- Pagamento e riscossione degli affitti;
- Uso di carte Probabilità/Imprevisto;
- Entrare e uscire dalla Prigione;
- Ipotecare proprietà;
- Timer della durata della partita. (opzionale)

Gestione punti fedeltà:

- Acquisizione di punti fedeltà riscuotendo affitti;
- Utilizzo dei punti fedeltà combinati ai soldi per pagare affitti e comprare proprietà.
 - Relazione soldi-punti fedeltà in base al livello:
 - Facile: 1€ equivale a 2 punti (4 punti per l'imprenditore);
 - Medio: 1€ equivale a 4 punti (8 punti per l'imprenditore);
 - Difficile: 1€ equivale a 6 punti (16 punti per l'imprenditore);

Gestione dei livelli di difficoltà:

- Scelta del livello di difficoltà (Facile, Medio, Difficile), con variazioni nei costi delle proprietà e conversione dei punti fedeltà.

Gestione multiplayer online:

- Sistema Client-Server per giocare online in tempo reale;
- Possibilità di avviare più partite contemporaneamente;
- Creazione di partite da parte di un admin con inviti per altri giocatori tramite codice.

1.4 REQUISITI NON FUNZIONALI

Prestazioni:

- Il sistema deve supportare un minimo di 2 e un massimo di 6 giocatori per partita;
- Deve essere in grado di gestire più partite simultaneamente senza compromettere le prestazioni.

Usabilità:

- L'interfaccia utente deve essere intuitiva e facile da usare, con istruzioni chiare per i giocatori;
- Deve includere un manuale utente per installazione e modalità di gioco.

Modularità:

- Il codice deve essere modulare per consentire futuri aggiornamenti.

Efficienza:

- Il tempo di risposta del server deve essere breve per operazioni comuni (esempio per movimento pedina).

Configurabilità:

- L'admin deve poter configurare facilmente le opzioni di gioco

1.5 CASI D'USO

Attori:

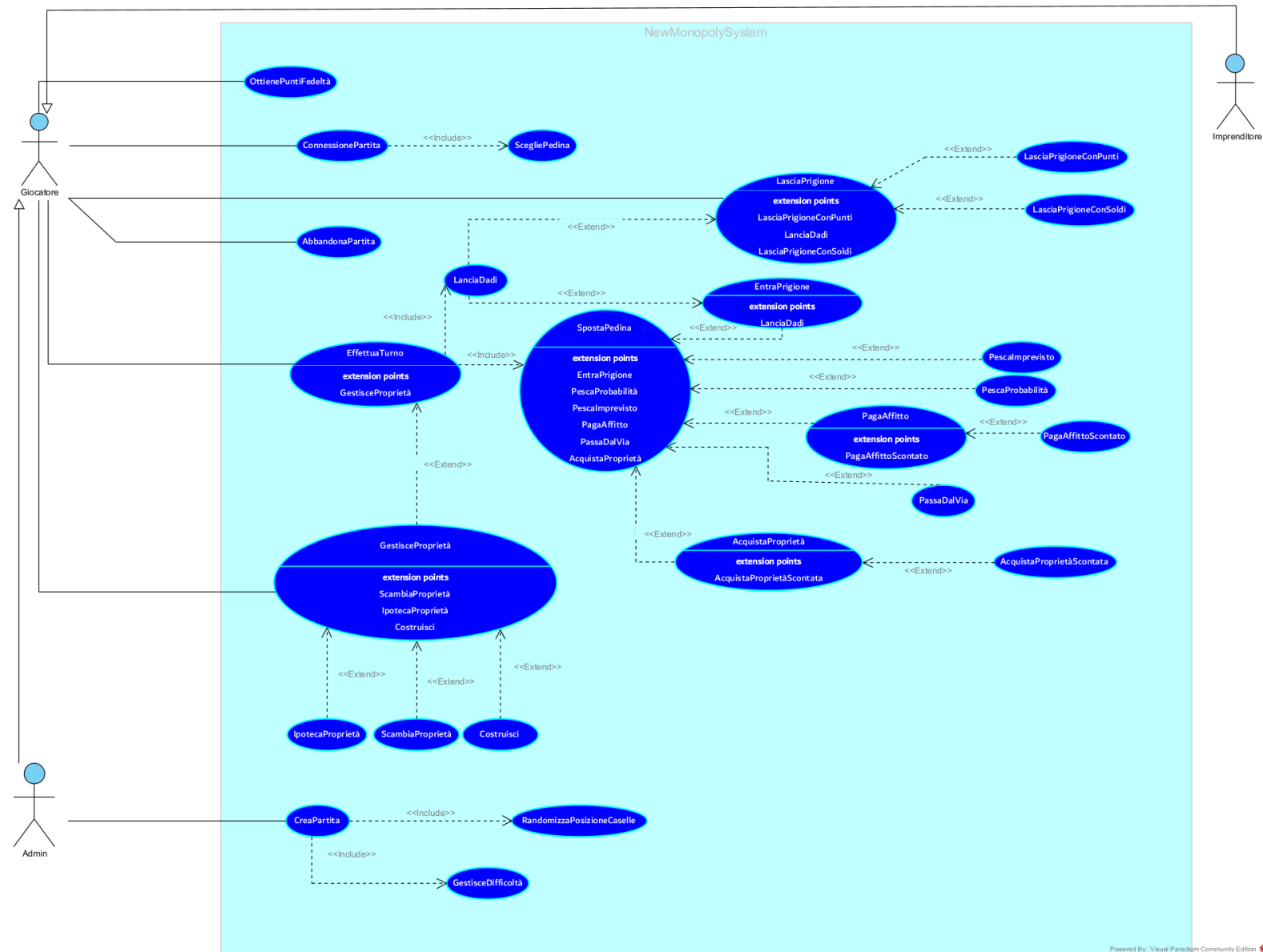
- **Giocatore:** interagisce con le funzionalità di gioco principali, come partecipare a una partita, lanciare i dadi, spostare la pedina, ecc.
- **Admin:** si occupa di configurare e creare nuove partite, definendo i parametri di gioco come la difficoltà e la disposizione casuale delle caselle.
- **Imprenditore:** ha le stesse funzionalità del giocatore, con la differenza che partecipa ad una partita dopo che essa è già iniziata.

Il Giocatore è una generalizzazione dell'Admin e dell'imprenditore.

Casi d'Uso principali:

- CreaPartita (funzionalità assegnata all'Admin): permette di configurare i livelli di difficoltà e di randomizzare le caselle, che di base sono posti a "Facile" e su "No".
- ConnessionePartita: permette di partecipare a una partita. Il giocatore sceglie la pedina e accede alla partita.
- EffettuaTurno: questo caso d'uso include:
 - LanciaDadi: lanciando i dadi puoi entrare in prigione e lasciare la prigione
 - SpostaPedina, permette il movimento della pedina,
 - gestisce diversi scenari:
 - EntraPrigione
 - PescaImprevisto
 - PescaProbabilità
 - PagaAffitto (con opzione di sconto tramite i punti fedeltà)
 - PassaDalVia
 - AcquistaProprietà (con opzione di sconto tramite i punti fedeltà)
- GestioneProprietà: che estende effettua turno, e gestisce diversi scenari:
 - IpotecareProprietà
 - ScambiaProprietà
 - Costituisci: per costruire case e alberghi
- AbbandonaPartita: un giocatore può abbandonarla in qualunque momento
- OttieniPuntiFedeltà: per prendere punti

1.6 DIAGRAMMA DEI CASI D'USO



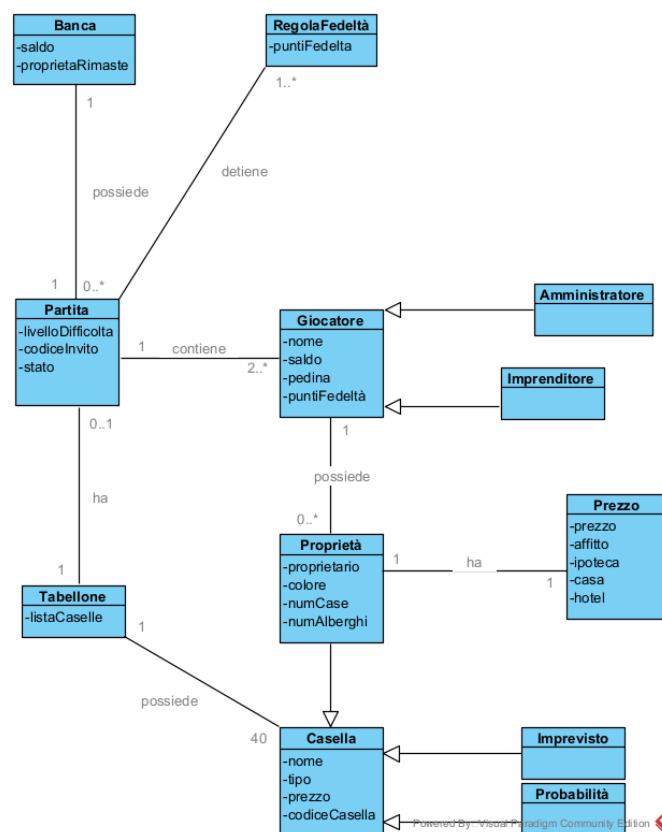
2 CAPITOLO: ANALISI

2.1 INTRODUZIONE

Durante la fase di analisi, i requisiti identificati vengono ulteriormente esplorati. Il suo scopo principale è fornire una visione chiara e completa del sistema da sviluppare, definendo i suoi componenti principali, le loro interazioni e il flusso delle operazioni. In particolare, verranno presentati diagrammi che aiutano la comprensione e la progettazione del sistema. Questo permette di chiarire la struttura e il comportamento del sistema, facilitando una transizione verso la progettazione e l'implementazione.

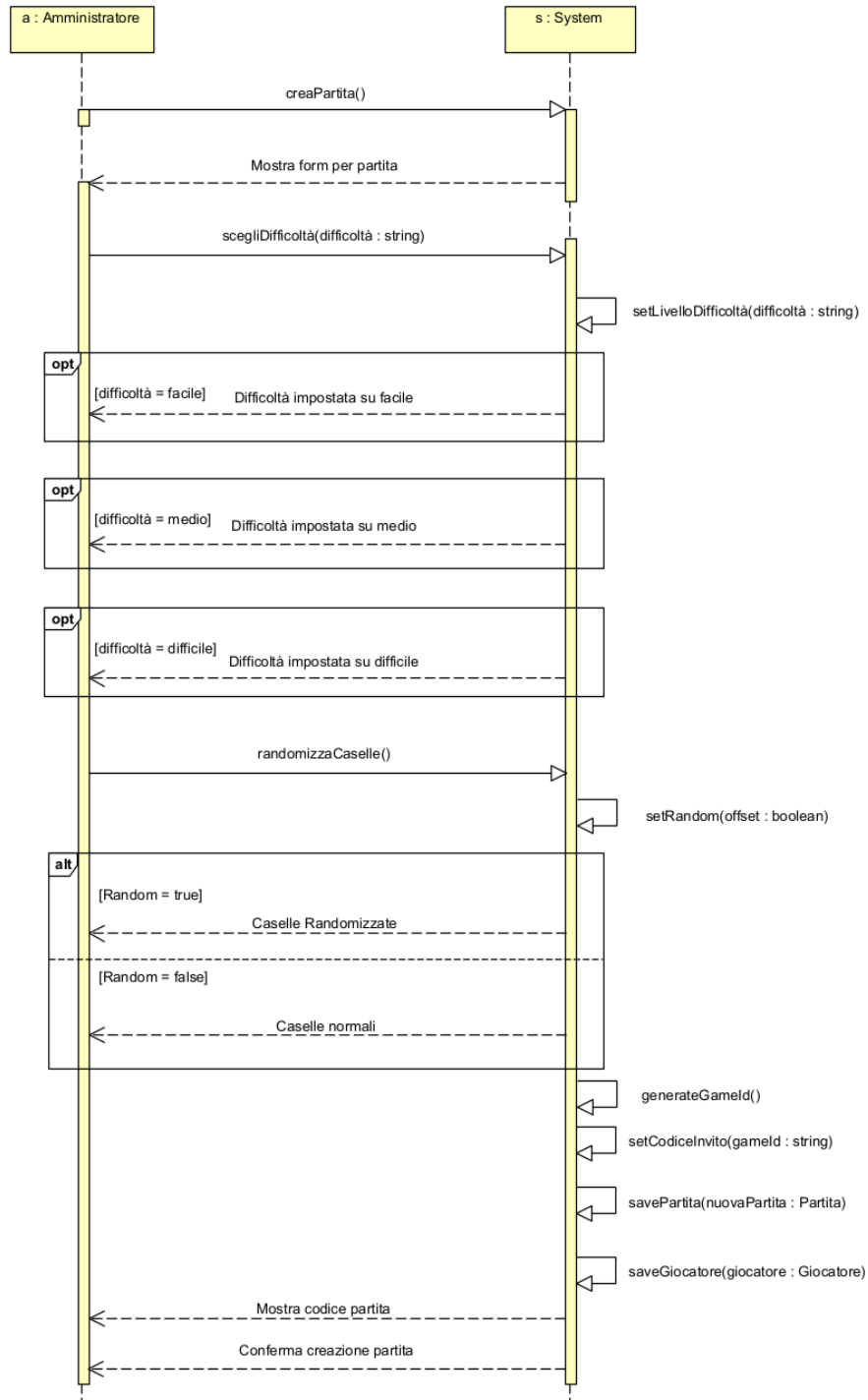
Abbiamo suddiviso il progetto in due iterazioni. Nella prima, ci siamo occupati delle funzionalità base del Monopoly, tra cui la creazione della partita, il lancio dei dadi, la gestione delle proprietà e il sistema di livello. Nella seconda iterazione, invece, abbiamo implementato funzionalità aggiuntive, come la randomizzazione delle caselle, l'introduzione della figura dell'imprenditore e il sistema dei punti fedeltà.

2.2 MODELLO DI DOMINIO

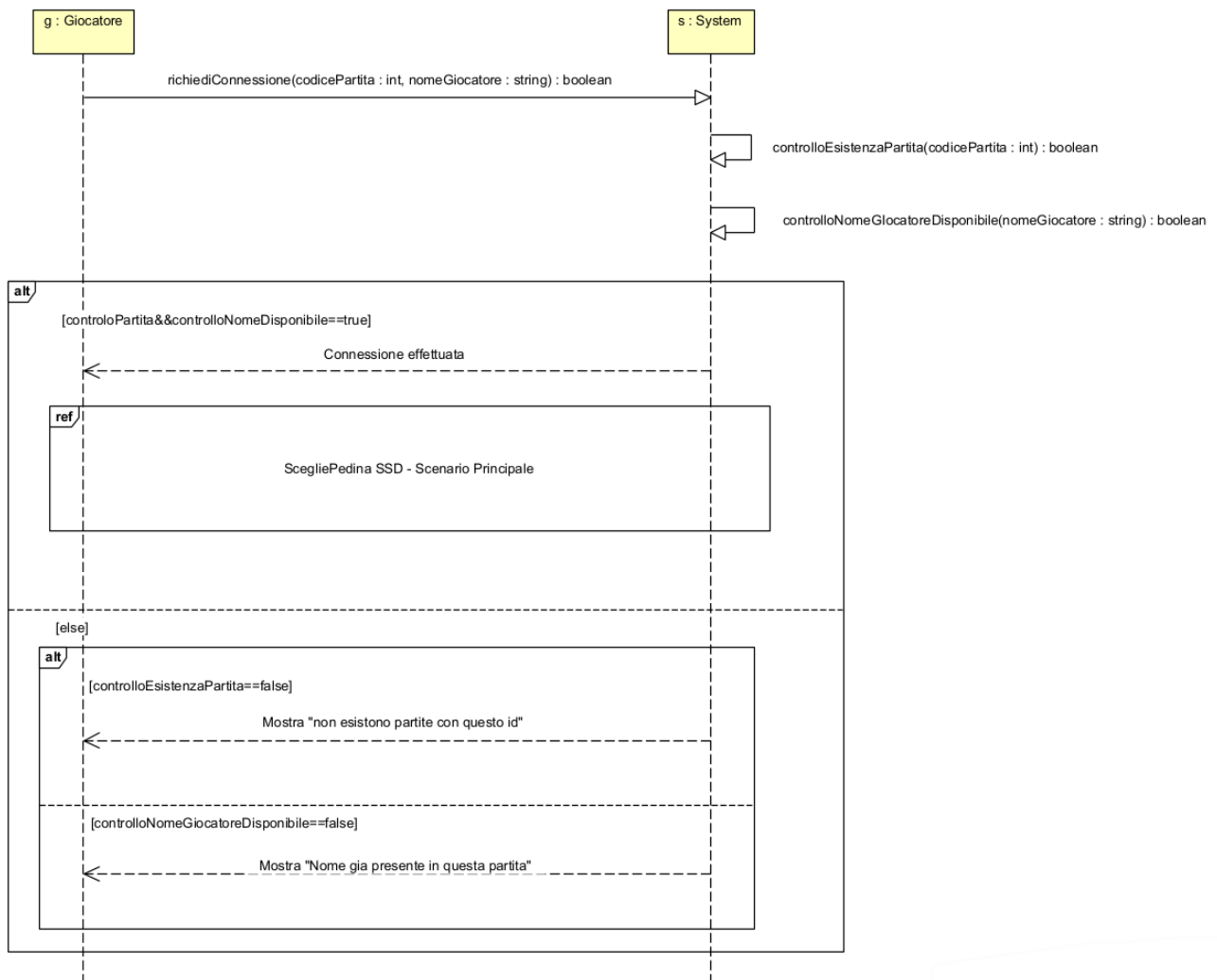


2.3 DIAGRAMMA DI SEQUENZA DI SISTEMA

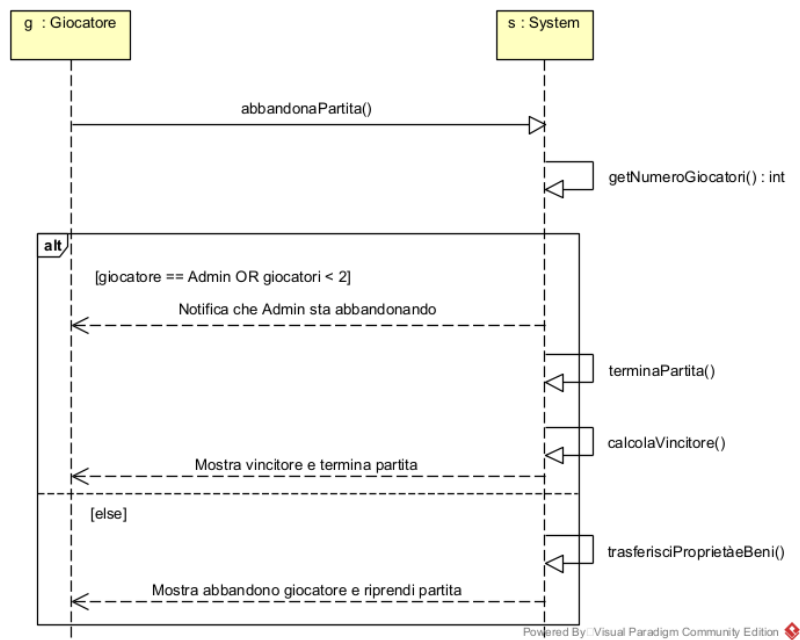
2.3.1 Caso d'Uso: CreaPartita



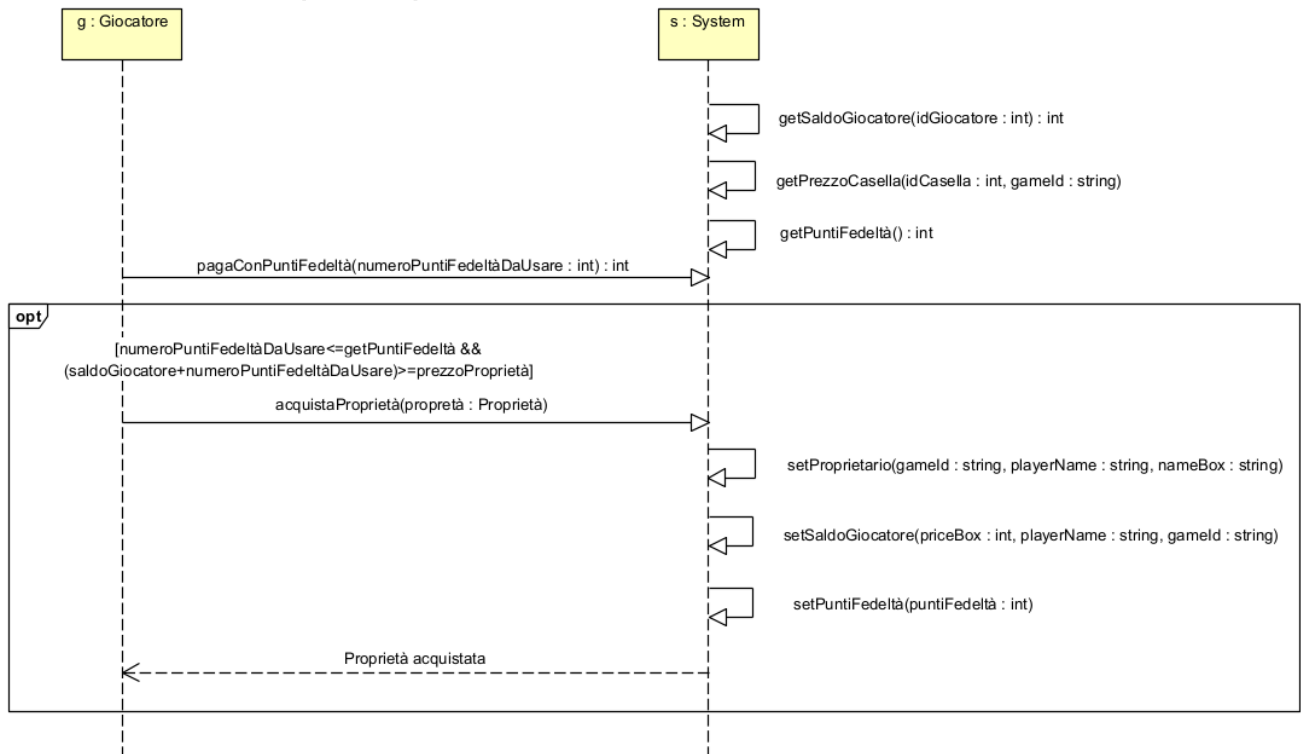
2.3.2 Caso d'Uso: ConnessionePartita



2.3.3 Caso d'Uso: AbbandonaPartita

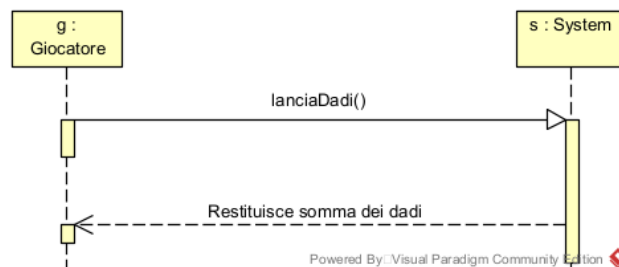


2.3.4 Caso d'Uso: AcquistaProprietà

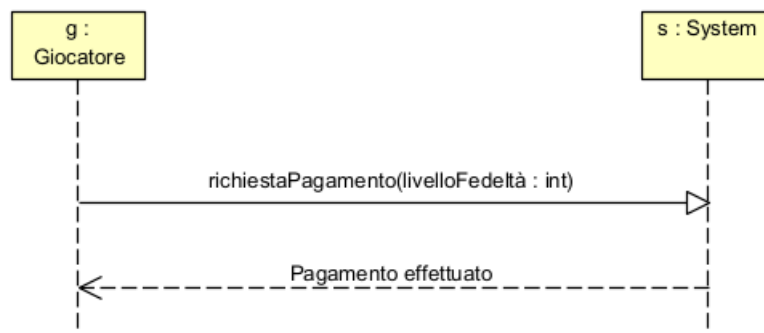


controllo punti fedeltà da backend: se $\text{saldoGiocatore} + \text{numeroPuntiFedeltàDaUsare} < \text{prezzoProprietà}$ va all'asta

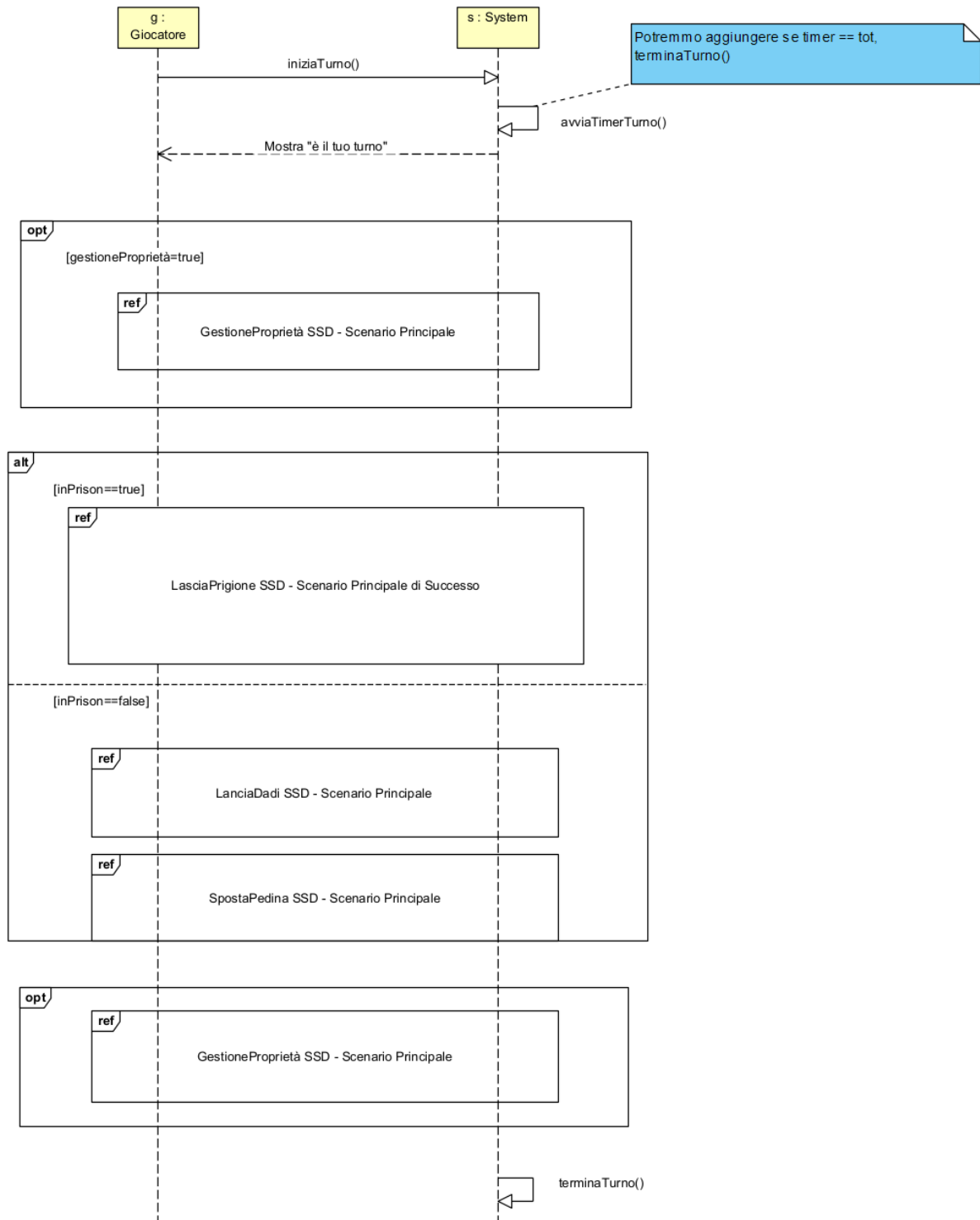
2.3.5 Caso d'Uso: LanciaDadi



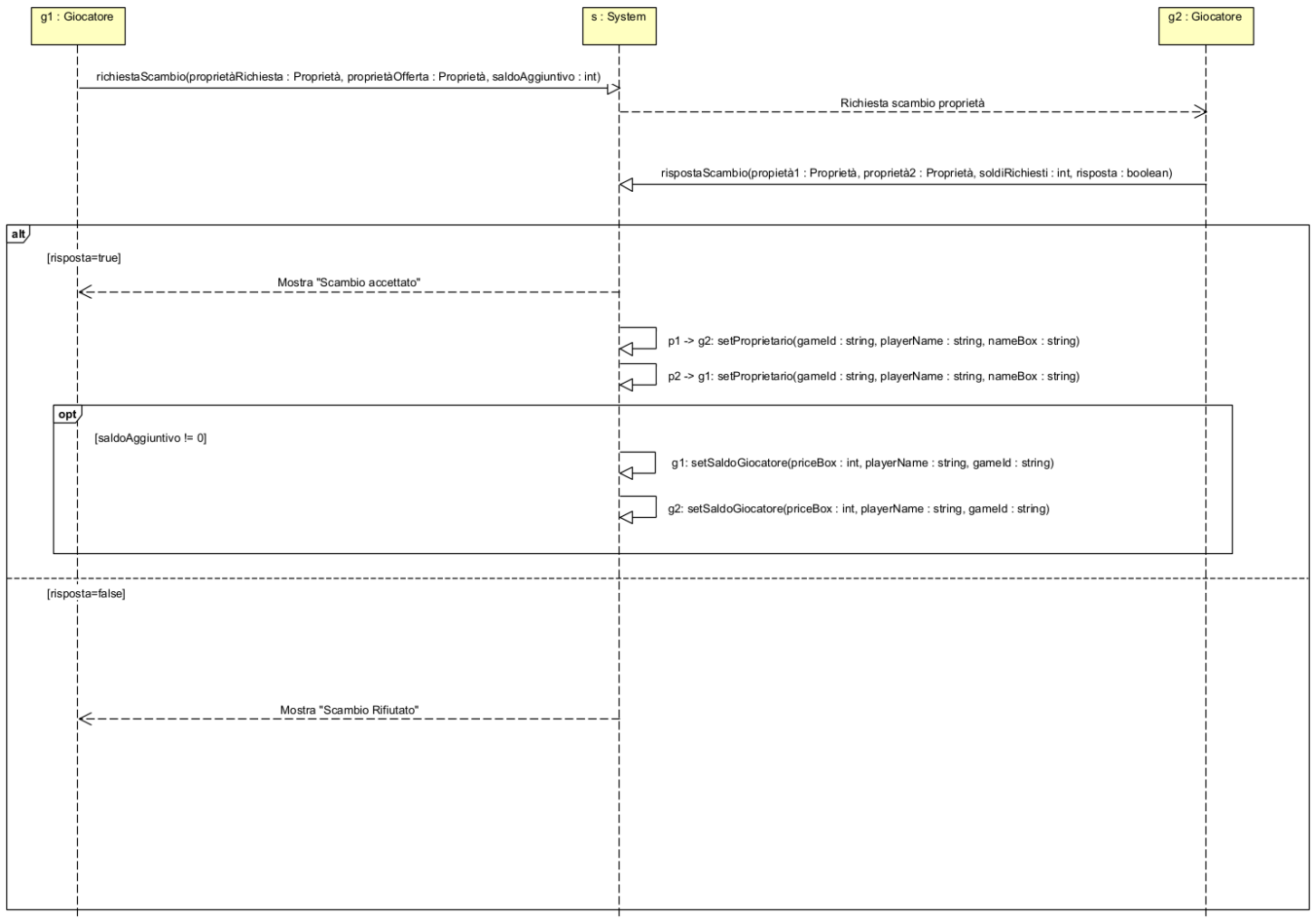
2.3.6 Caso d'Uso: PassaDalVia



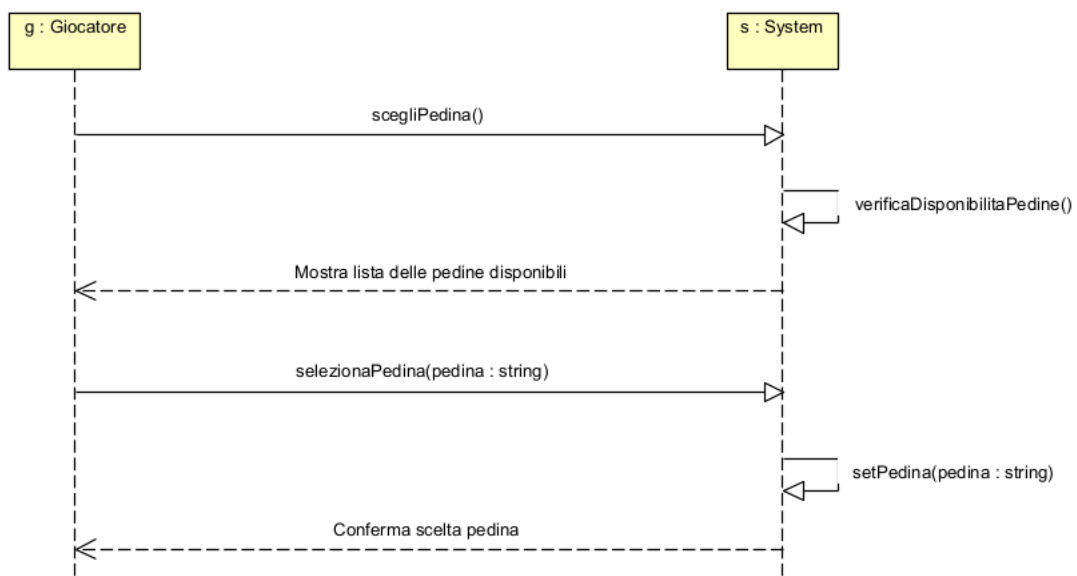
2.3.7 Caso d'Uso: EffettuaTurno



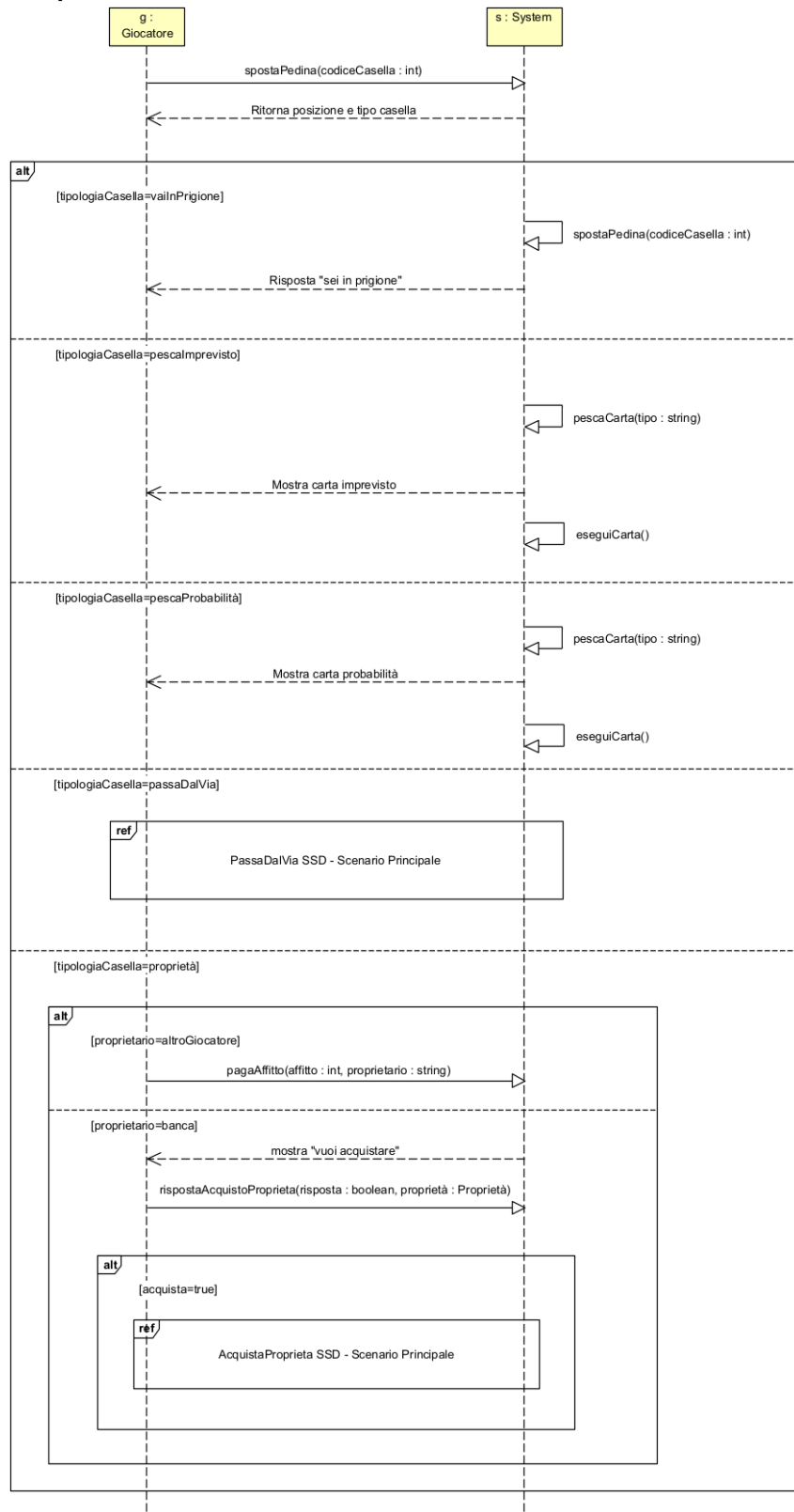
2.3.8 Caso d'Uso: ScambiaProprietà



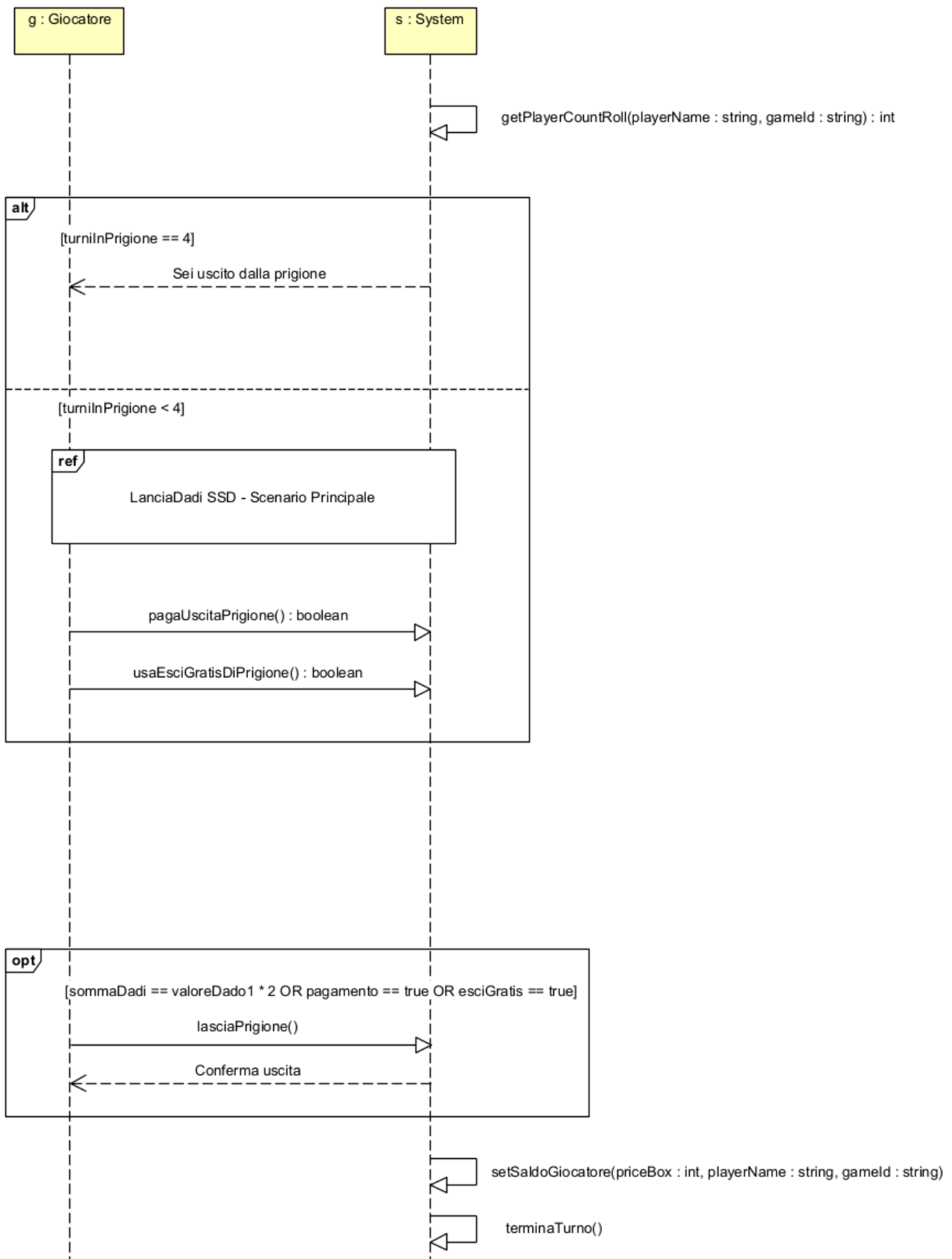
2.3.9 Caso d'Uso: ScegliPedina



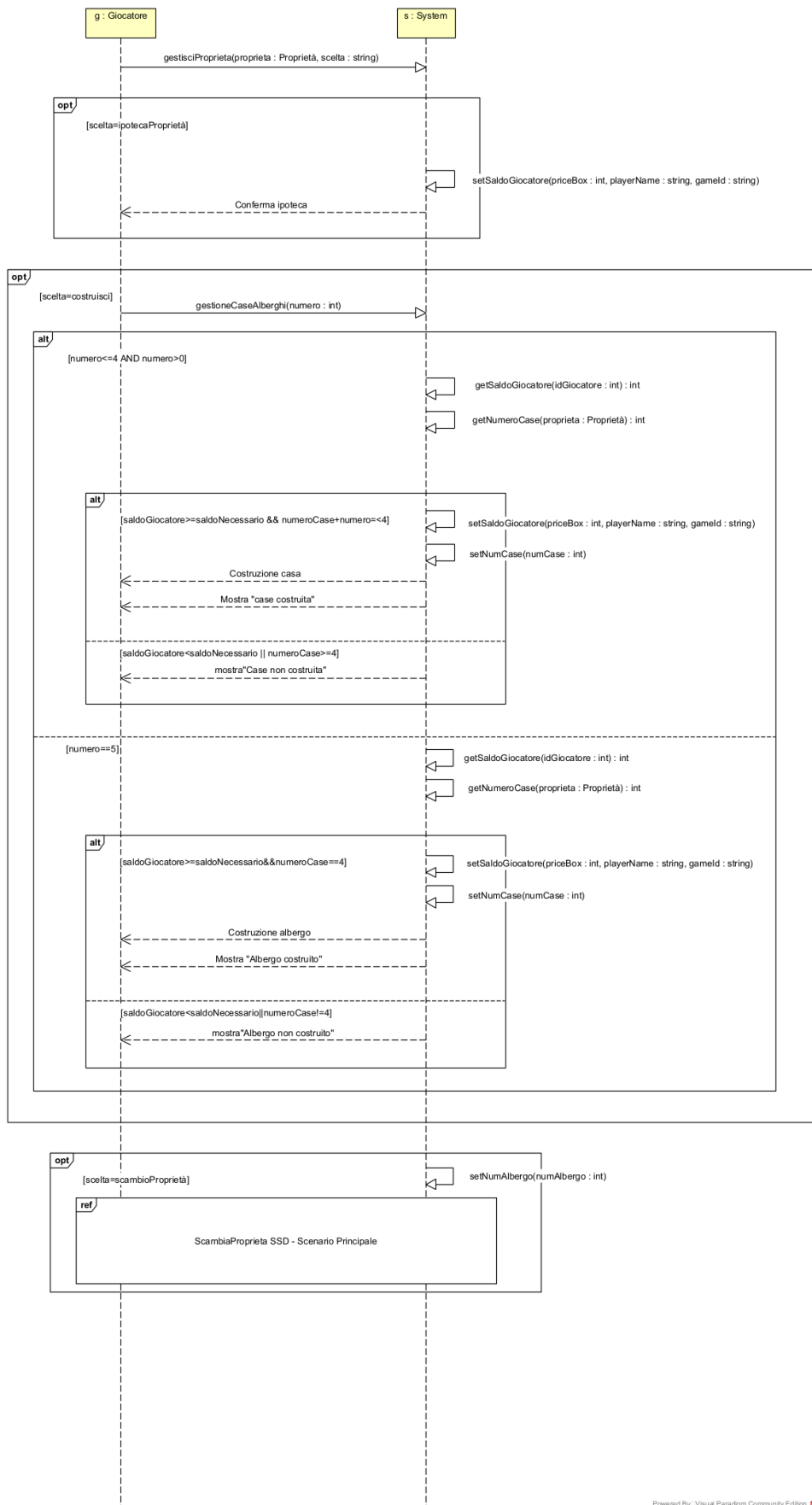
2.3.10 Caso d'Uso: SpostaPedina



2.3.11 Caso d'Uso: LasciaPrigione



2.3.12 Caso d'Uso: GestisciProprietà



3 CAPITOLO: PROGETTAZIONE

3.1 INTRODUZIONE

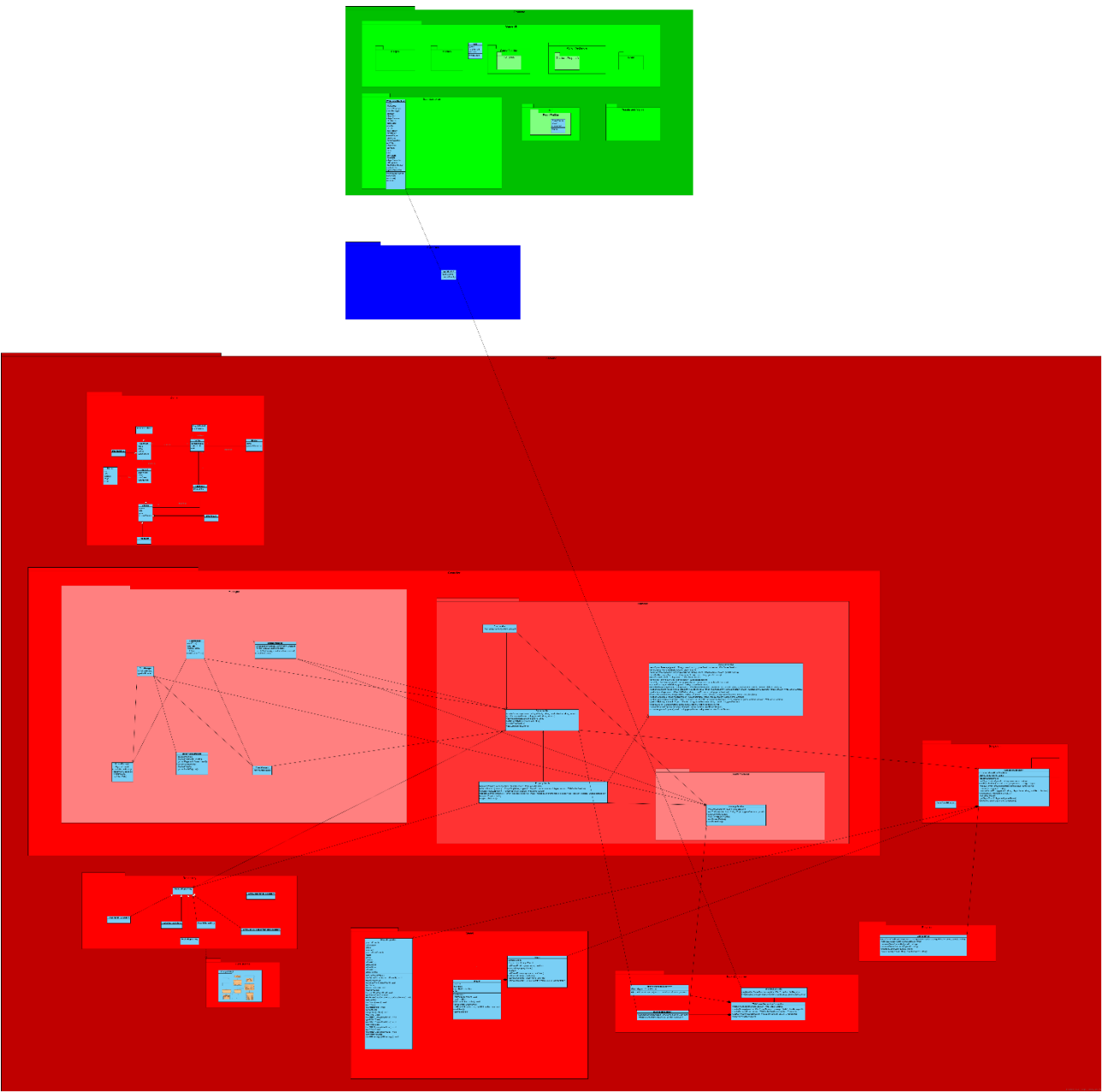
Per questo progetto abbiamo scelto di adottare un'architettura basata su un approccio **Event-Driven**. Questo significa che il flusso di lavoro è incentrato sugli eventi e sul loro trattamento.

Il frontend comunica con il backend utilizzando il protocollo **WebSocket**, che permette una comunicazione bidirezionale in tempo reale.

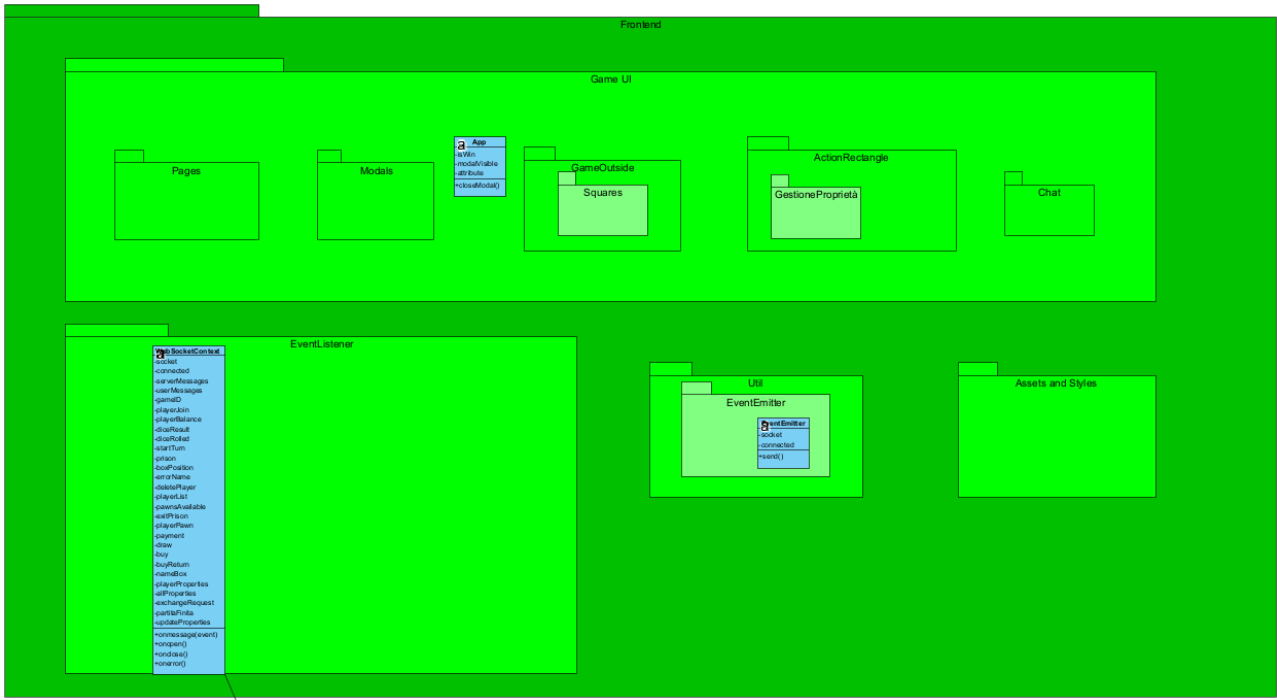
Quando il back-end riceve un messaggio, questo verrà intercettato dalla classe **WebSocketConnectorDispatcher**. Questa classe analizzerà il contenuto del messaggio ricevuto per inoltrarlo alle classi **Manager** o **Handler** pertinenti. Esse svolgono il loro compito, che può includere:

- Accesso al database: attraverso le **Repository**, per leggere o aggiornare i dati;
- Comunicazione con il front-end: utilizzando il **MessageService**, il quale può inviare risposte o notifiche al front-end.

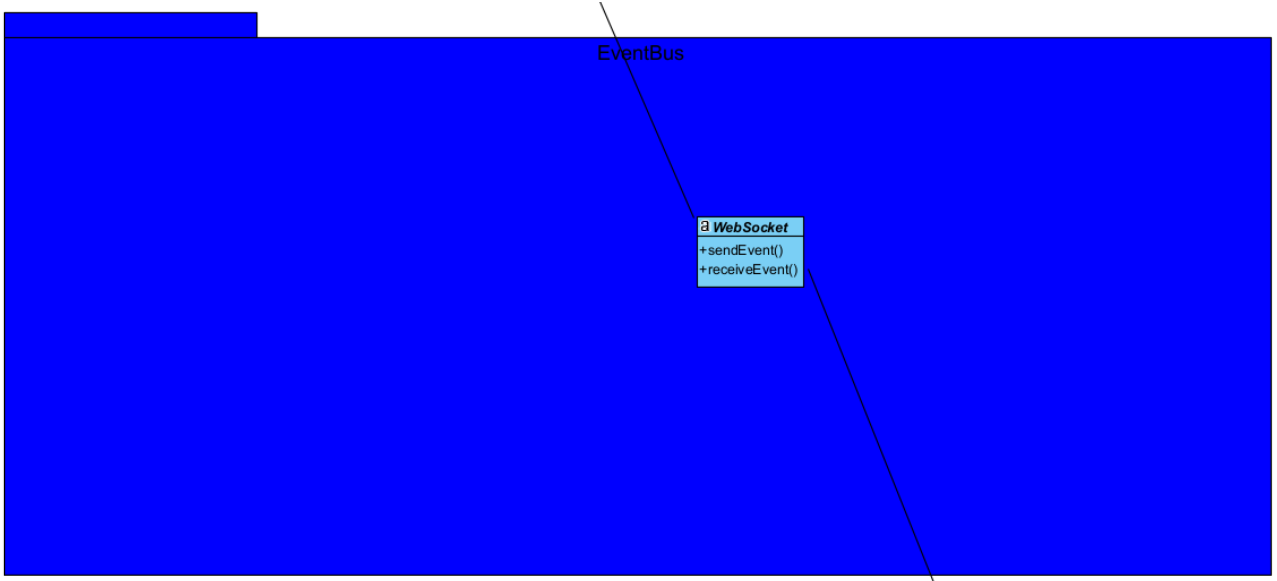
3.2 DIAGRAMMA DELL'ARCHITETTURA SOFTWARE



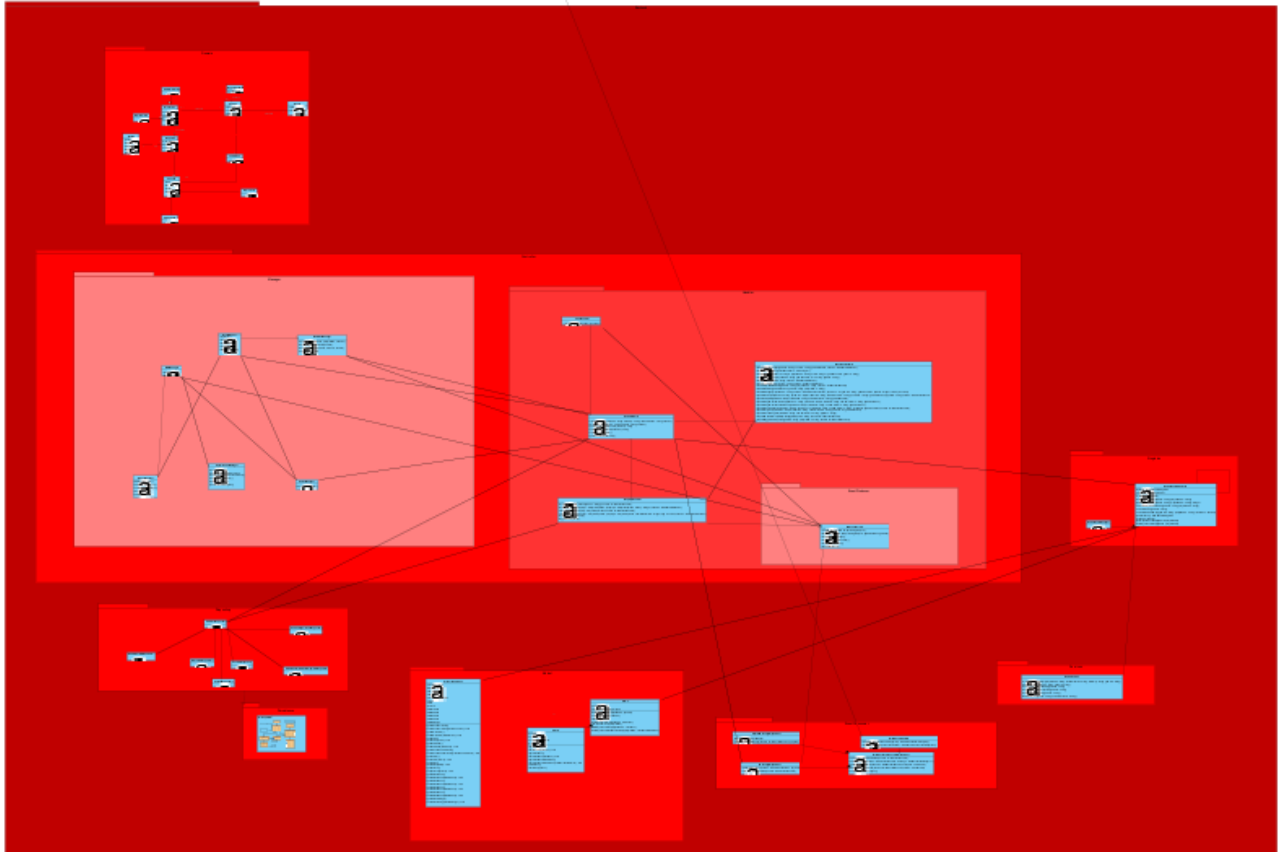
3.2.1 Frontend



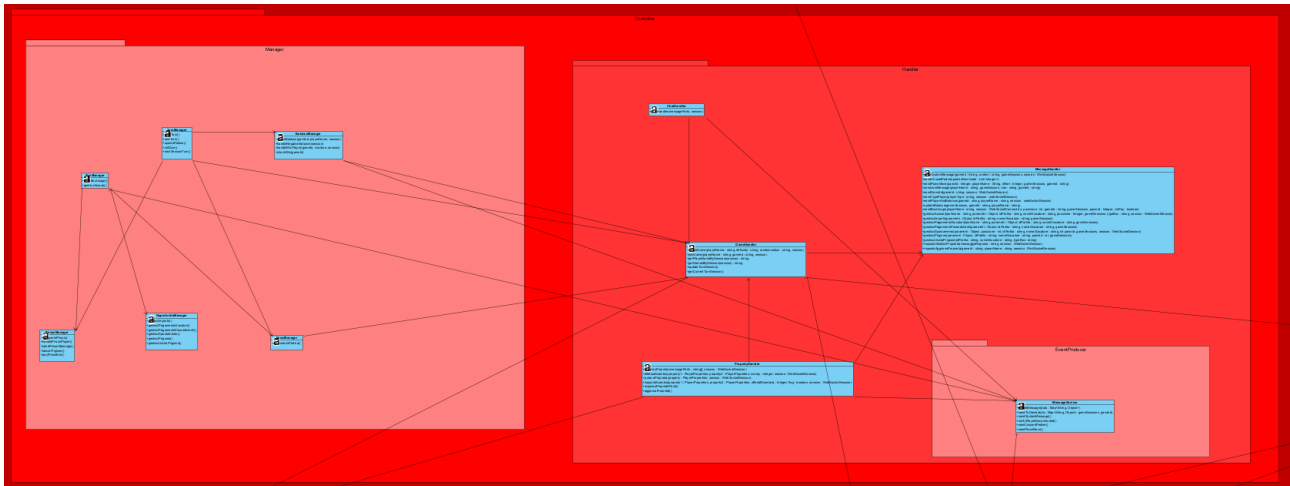
3.2.2 EventBus



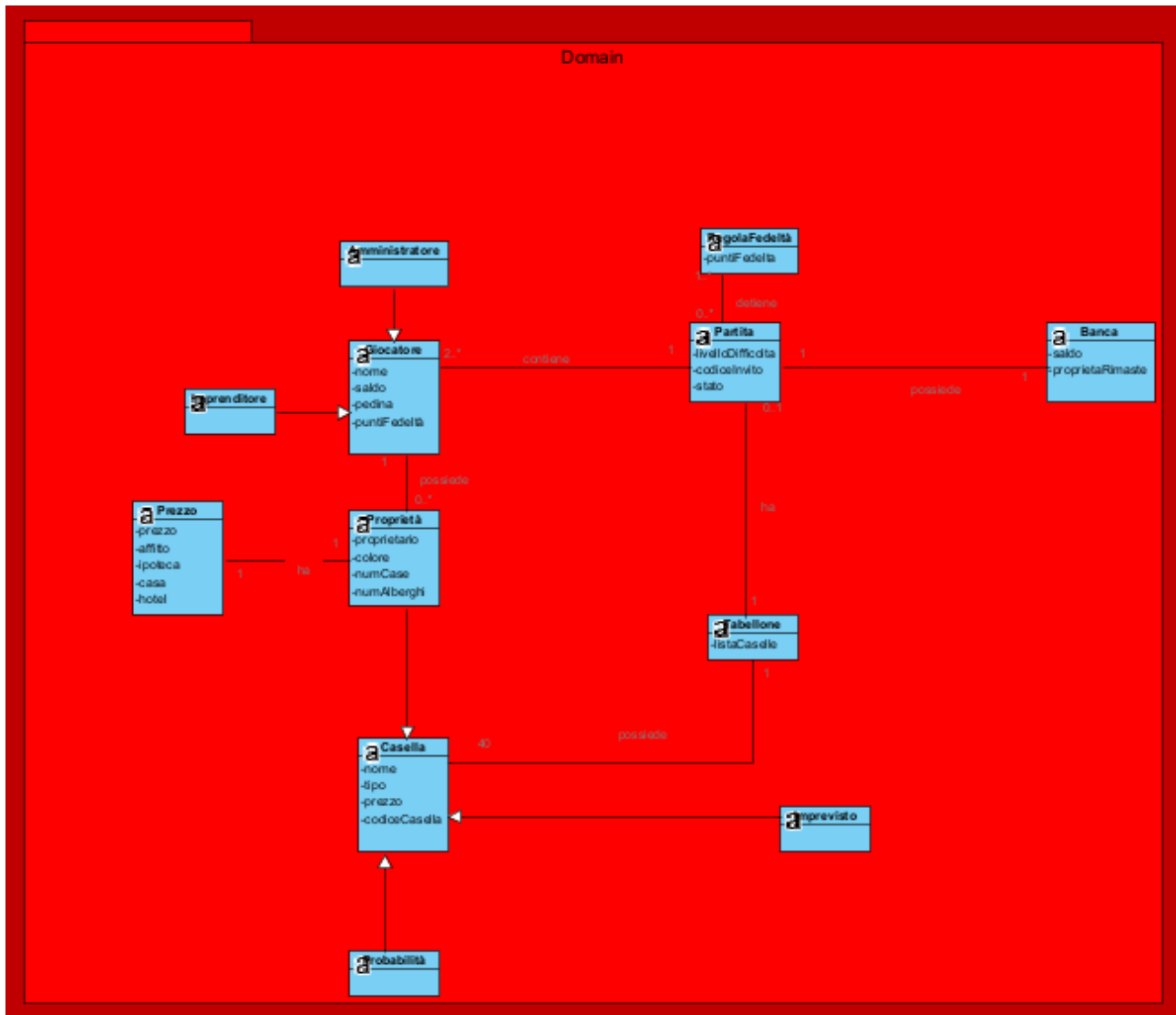
3.2.3 Backend



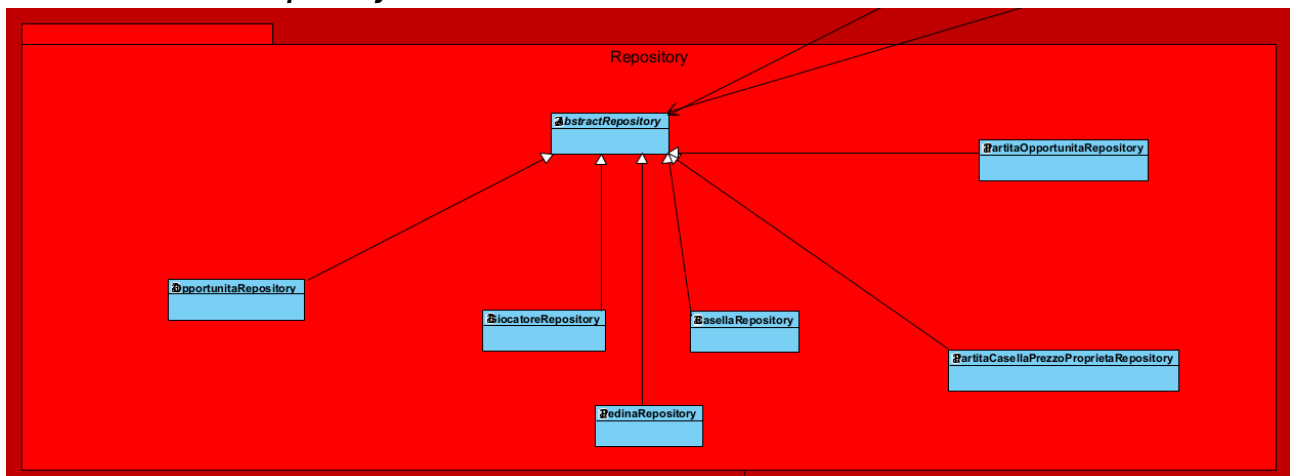
3.2.3.1 Backend – Controller



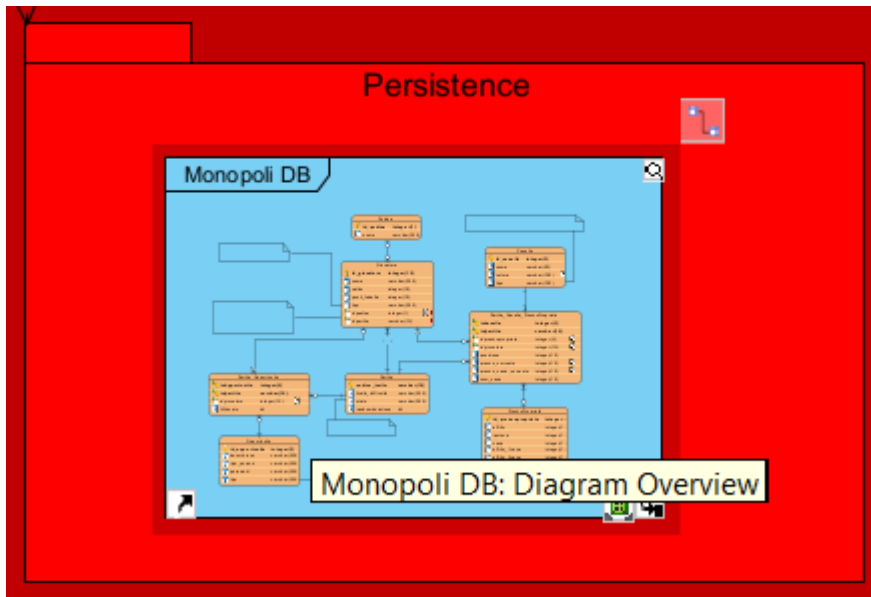
3.2.3.2 Backend – Domain



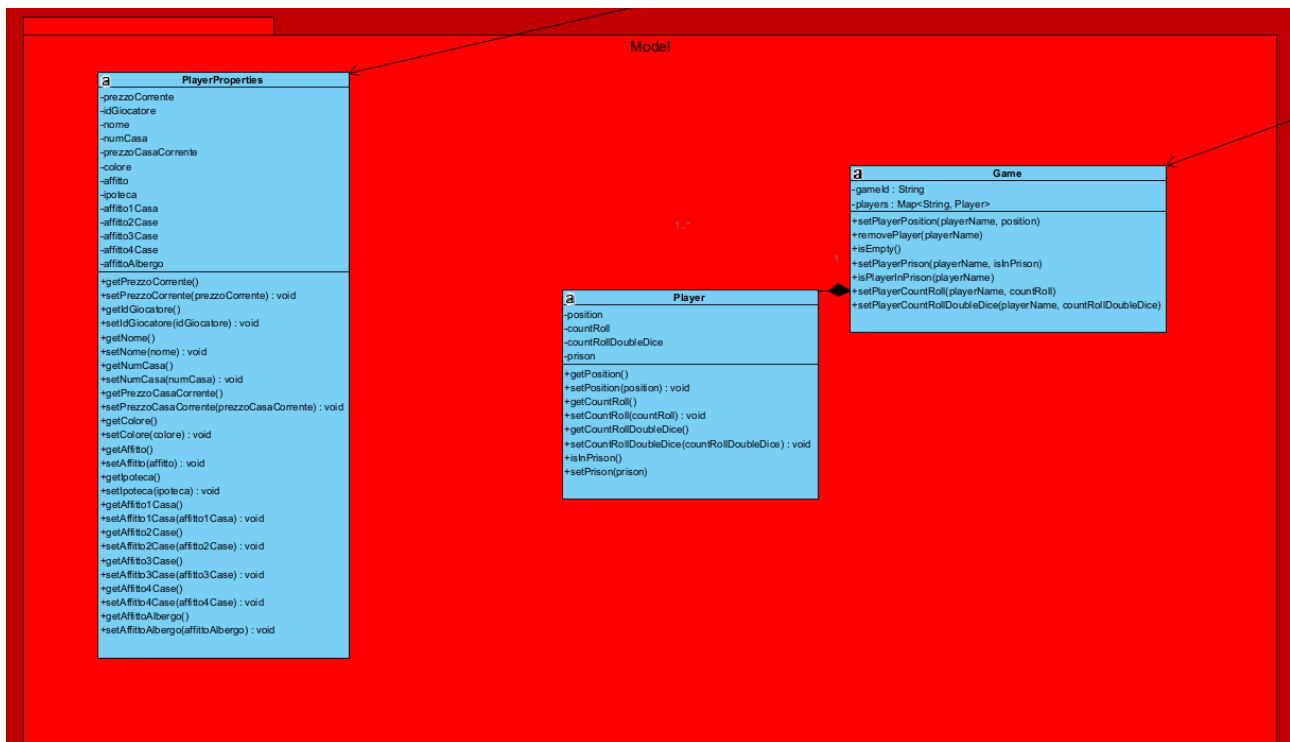
3.2.3.3 Backend – Repository



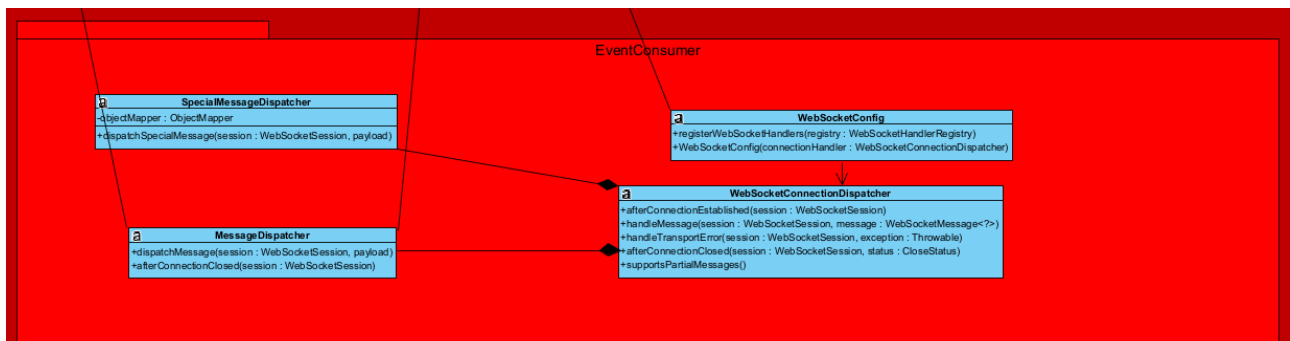
3.2.3.4 Backend – Persistence



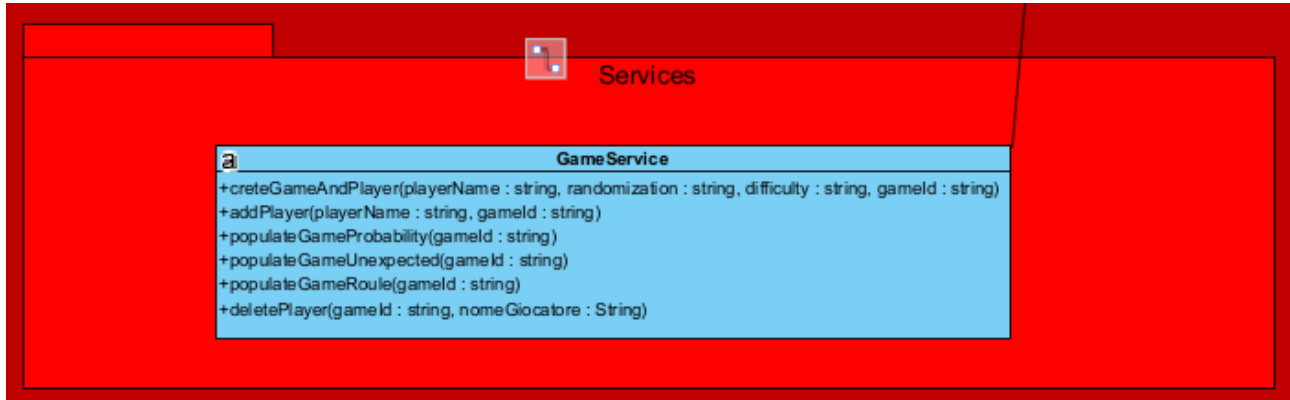
3.2.3.5 Backend – Model



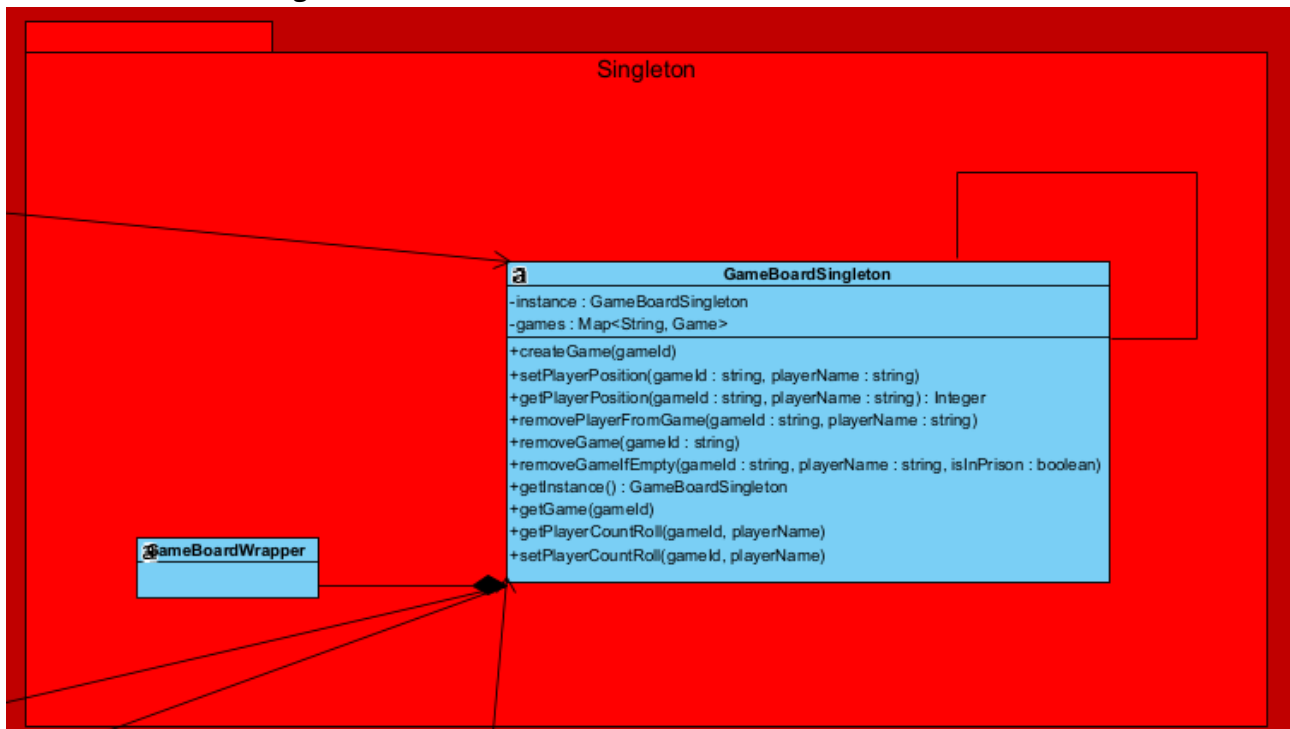
3.2.3.6 Backend – EventConsumer



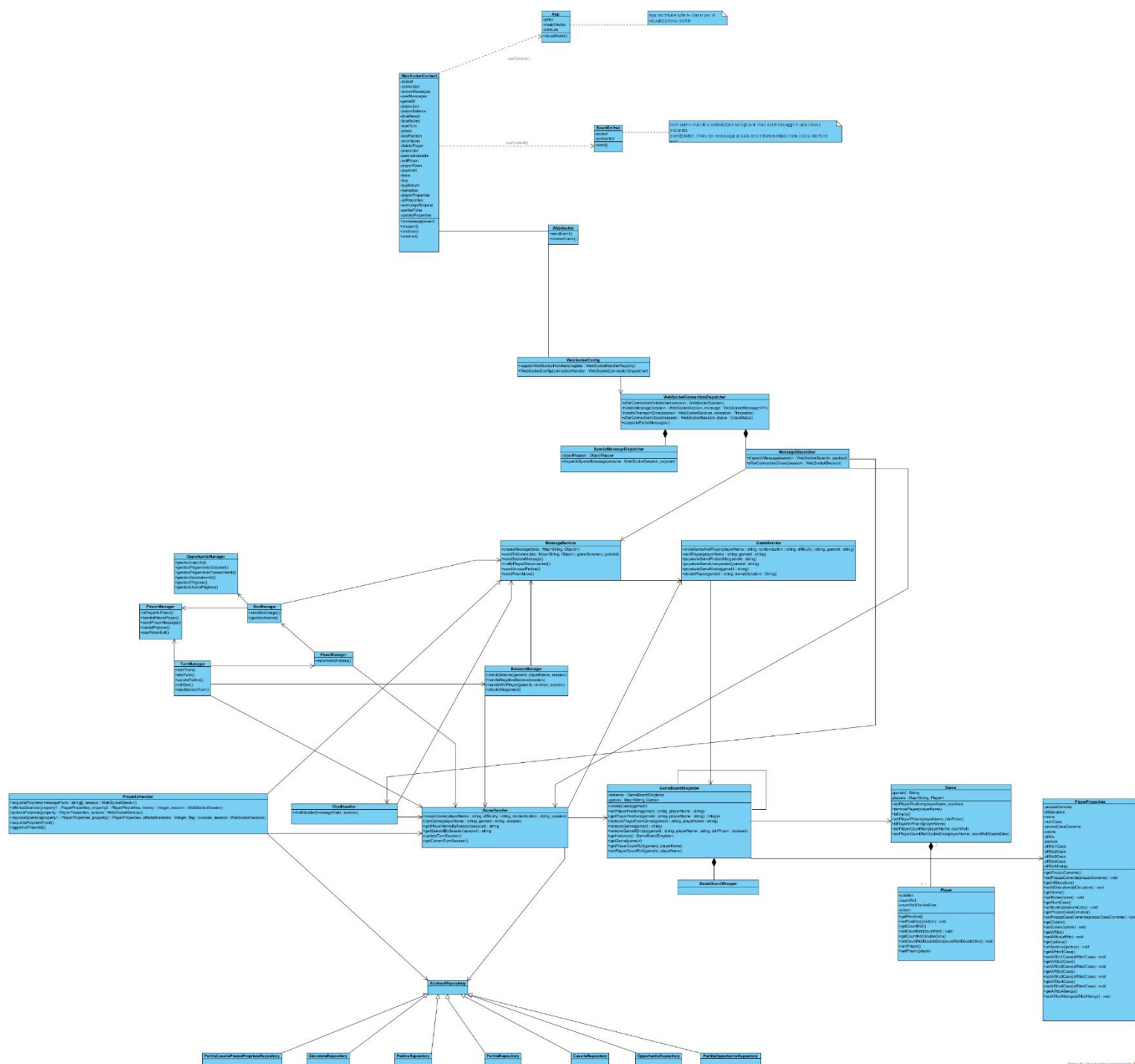
3.2.3.7 Backend – Service



3.2.3.8 Backend – Singleton

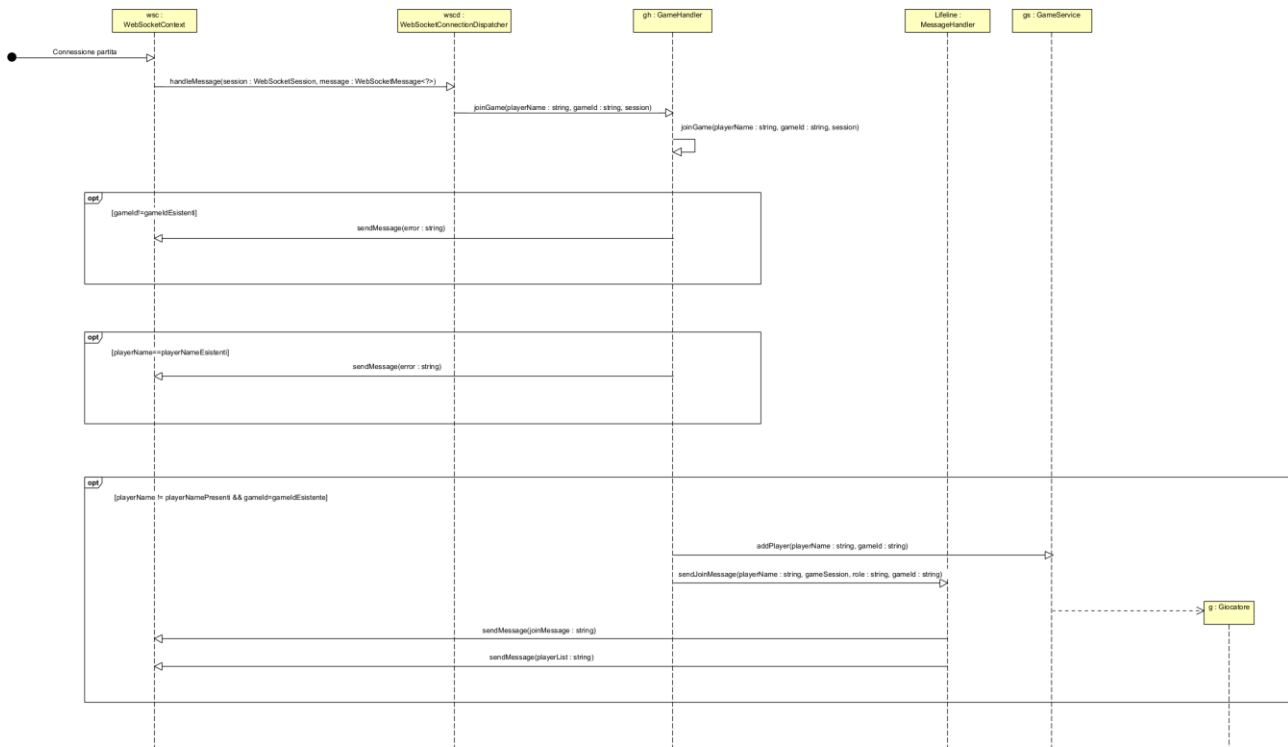


3.3 DIAGRAMMA DELLE CLASSI A LIVELLO DI PROGETTAZIONE

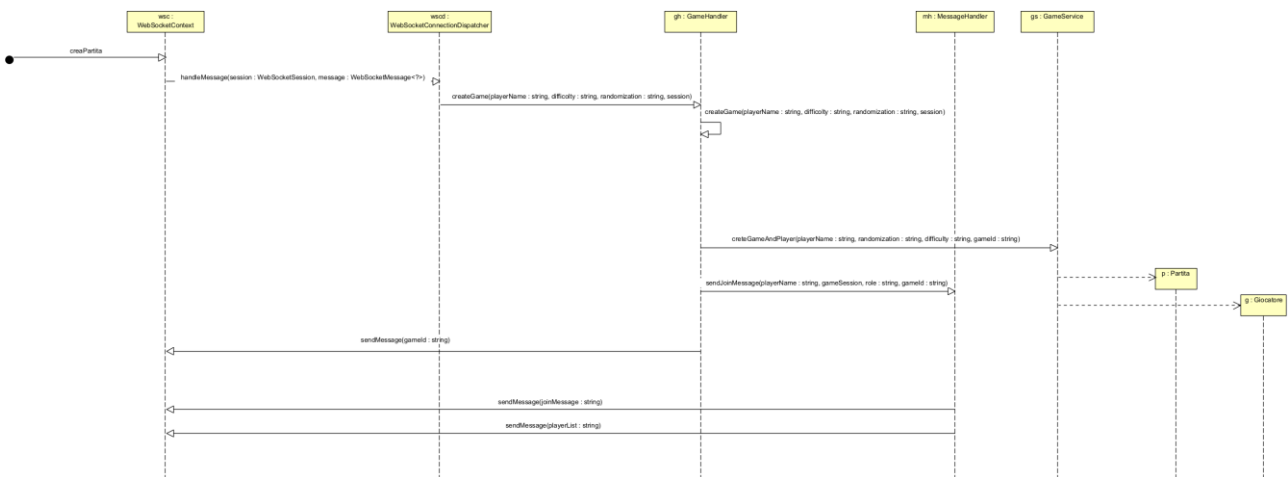


3.4 DIAGRAMMA DI INTERAZIONE: DIAGRAMMI DI SEQUENZA

3.4.1 Connessione partita

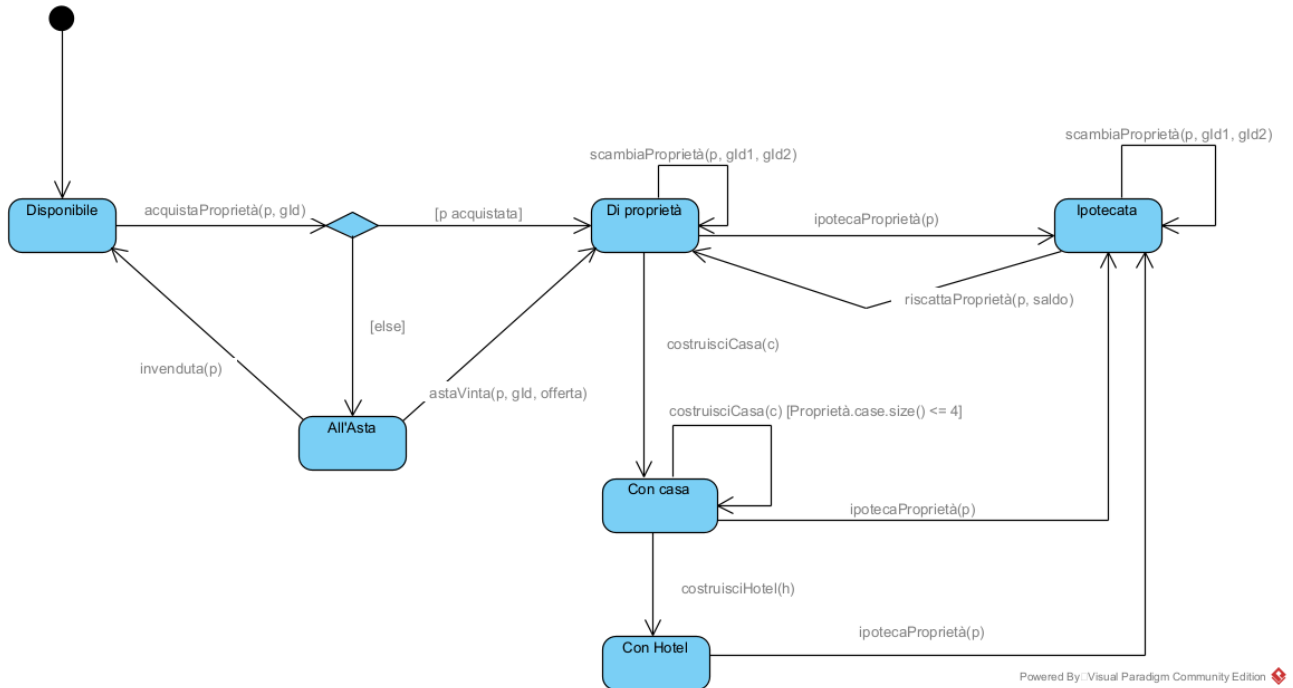


3.4.2 Crea partita

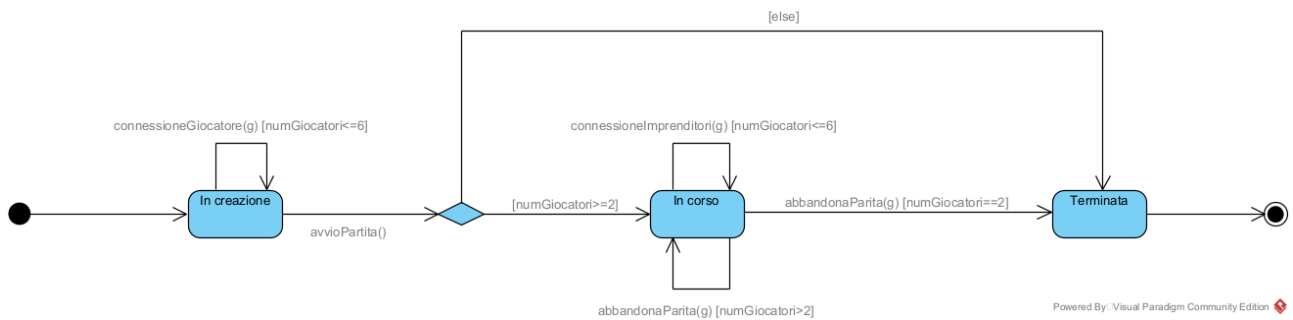


3.5 DIAGRAMMA A STATI

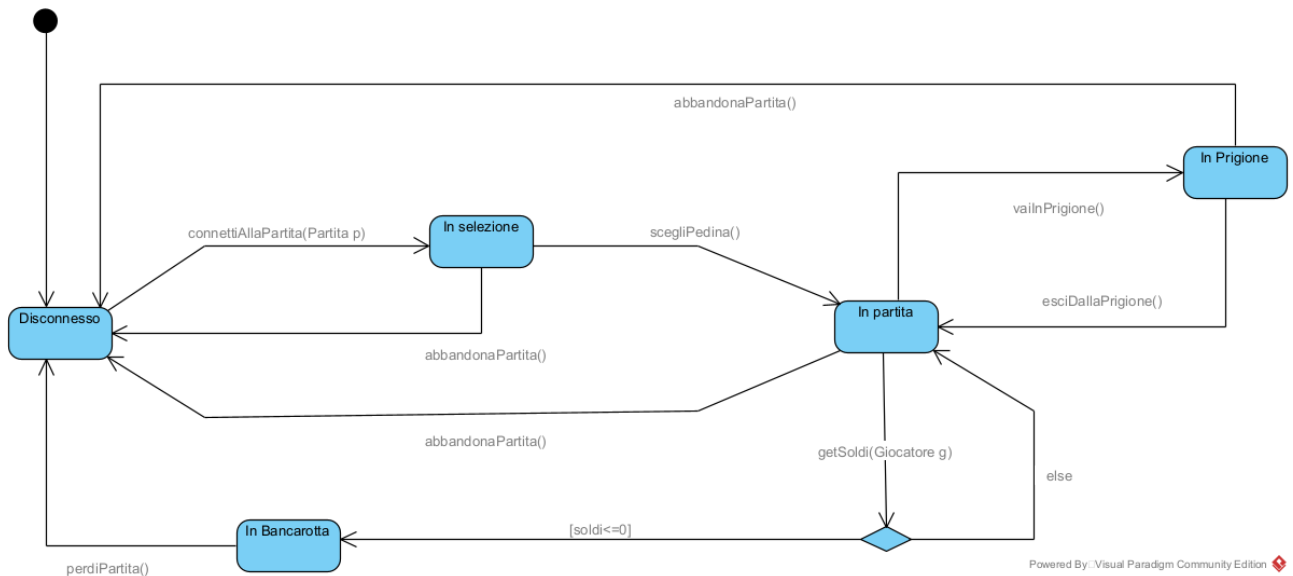
3.5.1 Proprietà



3.5.2 Partita

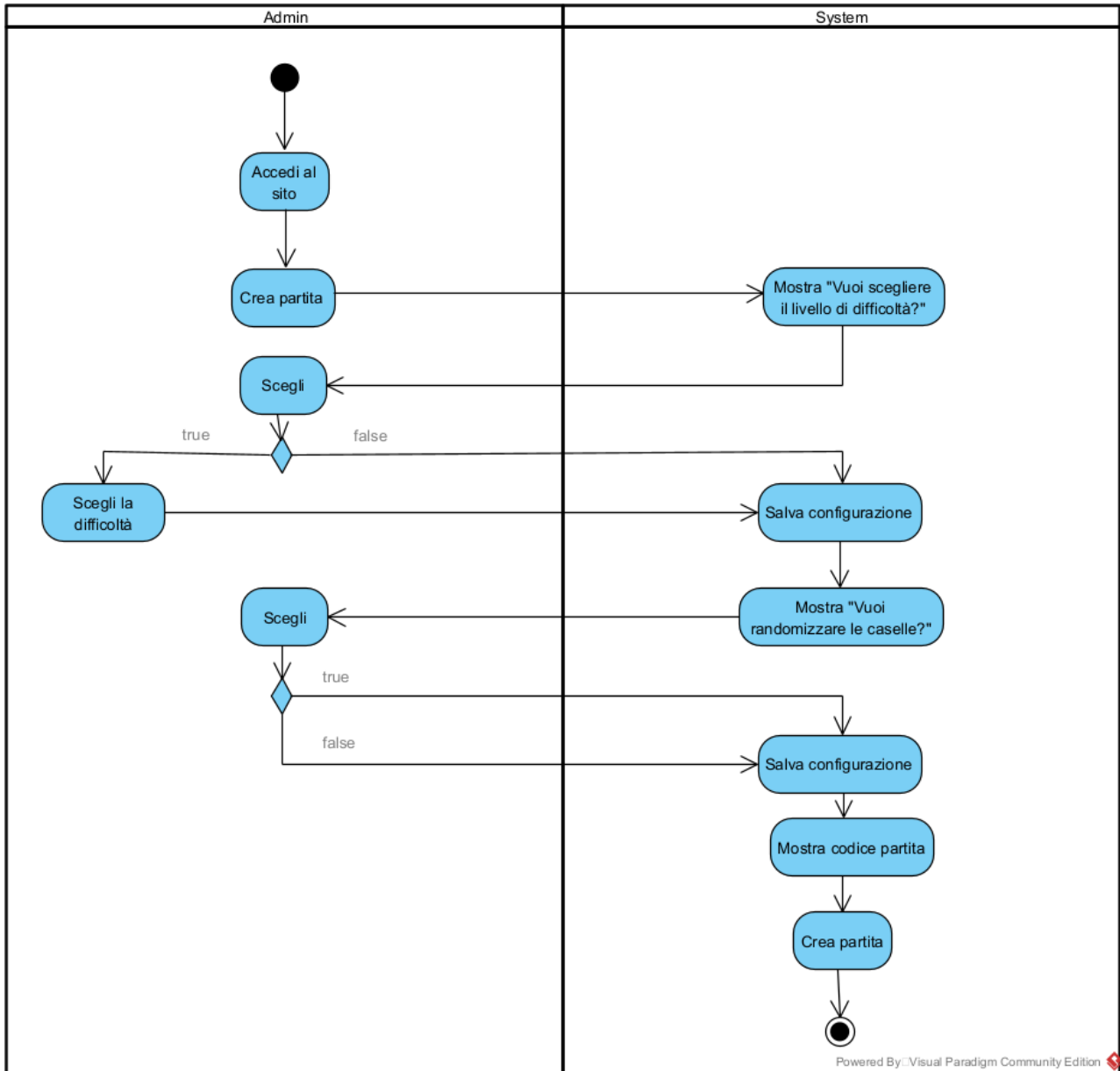


3.5.3 Giocatore

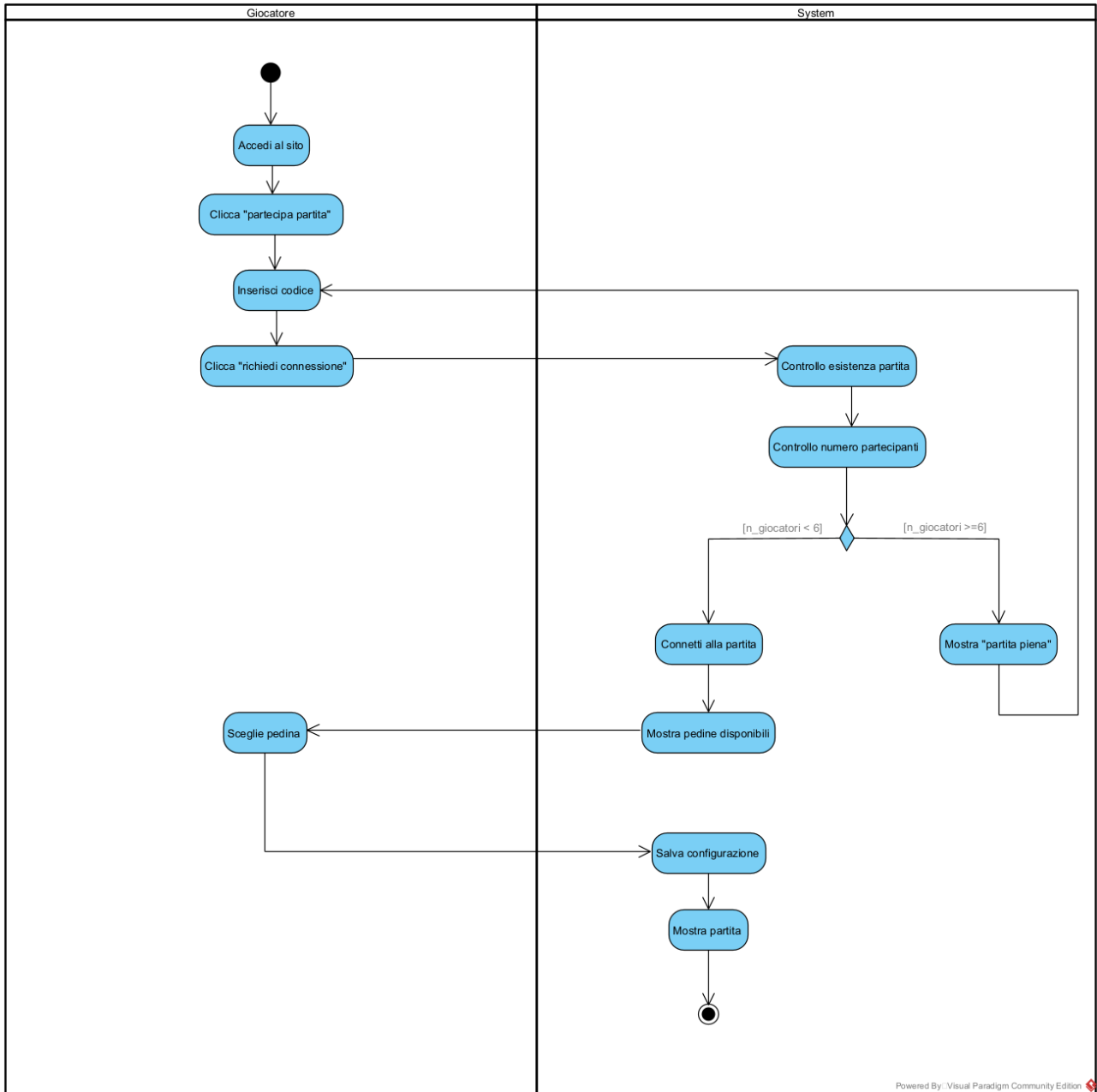


3.6 DIAGRAMMA DELLE ATTIVITÀ

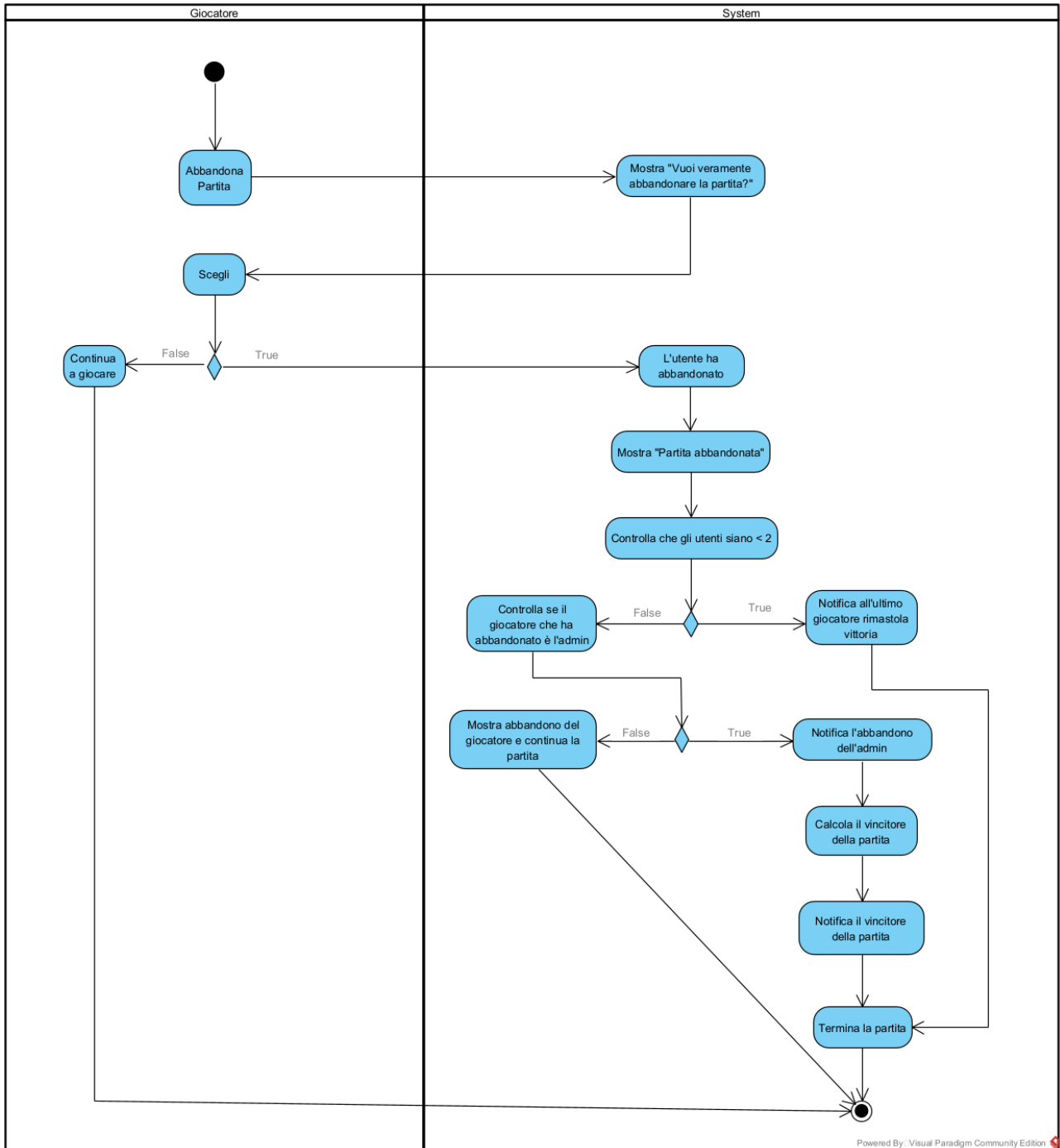
3.6.1 Crea Partita



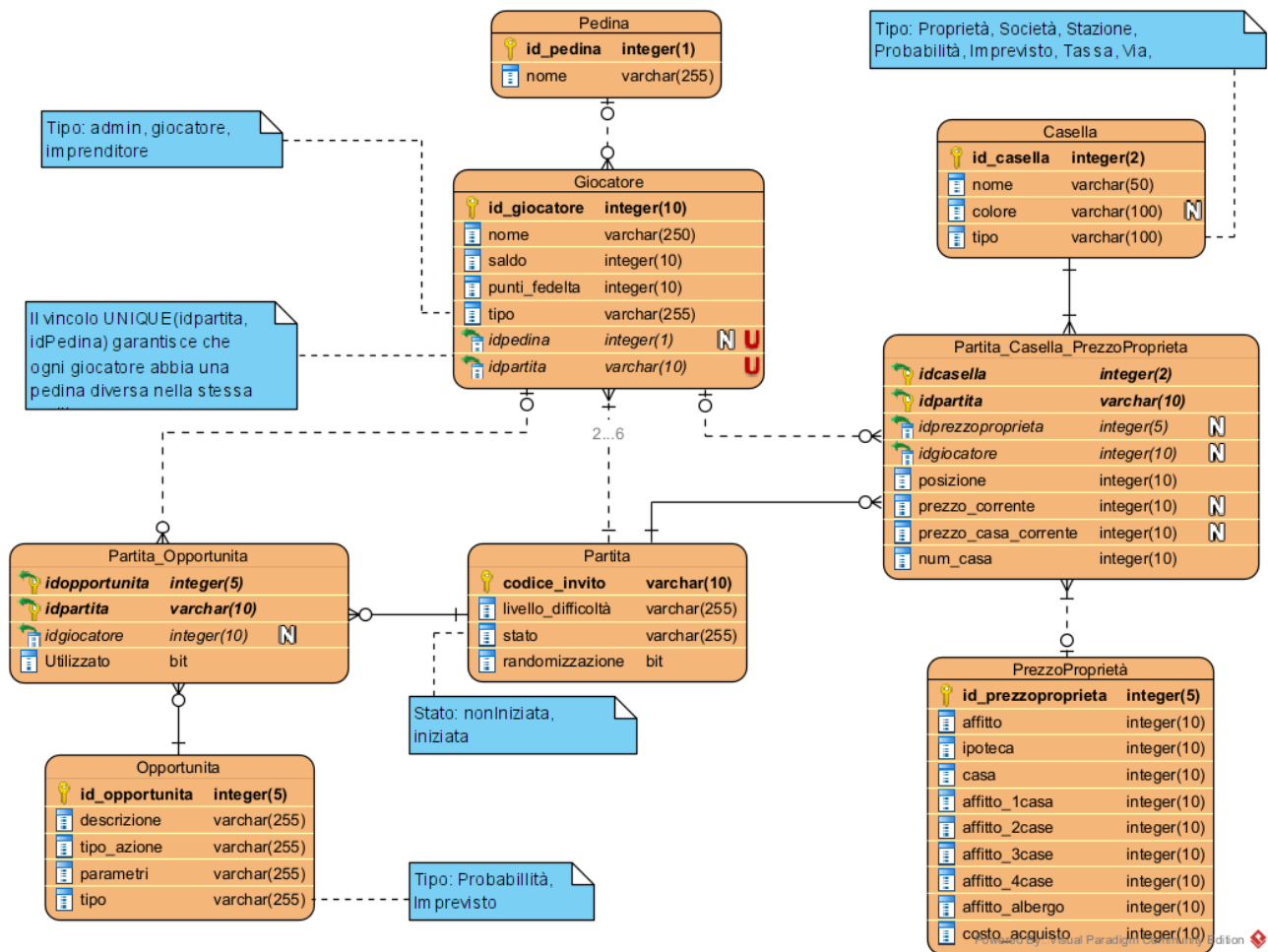
3.6.2 Connessione Partita



3.6.3 Abbandona Partita



3.7 DIAGRAMMA UML DEL DATABASE



Il Database è progettato per tutte le partite di gioco, distinguibili attraverso il campo *codice_invito* nella tabella **Partita**. Ogni partita è caratterizzata da uno *stato* (non iniziata, iniziata) e da un'opzione di randomizzazione che influenza la configurazione delle caselle di gioco. Lo stato è necessario per capire se l'utente che si aggiunge è un giocatore normale o un imprenditore.

Le seguenti tabelle statiche, che non subiscono modifiche durante il gioco:

- **Casella:** contiene i dati delle 40 caselle di gioco. Ogni casella è identificata da un *id_casella*, ha un *nome*, un *colore* (che è NULL per le caselle come Stazione o Via) e un *tipo* (es. Proprietà, Stazione, ecc.)
- **PrezzoProprieta:** fornisce i dettagli dei costi associati alle caselle di tipo **Proprietà**, **Stazione** e **Società**, inclusi:
 - Il costo di acquisto della casella
 - Gli affitti in base al numero di case costruite per le proprietà o il numero di stazioni/società possedute
 - L'associazione con casella corrispondente è effettuata tramite il campo *id_prezzoproprieta*, che coincide con l'*id_casella* corrispondente quando la randomizzazione è disabilitata.
- **Pedina:** contiene l'elenco delle pedine selezionabili dai giocatori.
- **Opportunità:** definisce le carte di probabilità e imprevisti. Ogni carta è descritta attraverso:
 - *Tipo_azione:* azione associata alla carta (tipo ricevi_importo oppure vai_avanti)

- *Parametri*: un JSON che dettaglia i valori dell'azione (es. "importo: 10", "id_casella=4")
- *Tipo*: specifica se si tratta di una carta Probabilità o Imprevisto.

La tabella **Partita_Casella_Prezzoproprieta** rappresenta la relazione delle caselle e del prezzo in una specifica partita. La popolazione di questa tabella dipende dalla modalità di randomizzazione.

Quando non c'è la randomizzazione, viene eseguita una query per popolare la tabella:

- Tutte le caselle mantengono la loro posizione originale (*id_casella*).
- Le caselle di tipo Proprietà, Stazione e Società vengono associate al relativo *id_prezzoproprieta*.
- Le altre caselle non hanno un valore associato per *id_prezzoproprieta* quindi verranno settate a NULL.

Quando c'è la randomizzazione, viene eseguita una query più complessa per randomizzare posizione e prezzi delle caselle di tipo Proprietà:

1. Si selezionano le caselle non di proprietà e si assegnano posizione fisse, mantenendo l'eventuale associazione con *id_prezzoproprieta*.
2. Si identificano le posizioni già occupate dalle caselle non proprietà (*numeri_occupati*) e si generano le posizioni libere (*numeri_disponibili*).
3. Le caselle di tipo Proprietà vengono ordinate casualmente e assegnate alle posizioni libere in modo randomico.
4. I prezzi delle proprietà vengono randomizzati e associati alle caselle.
5. Infine, le caselle non proprietà e quelle di proprietà vengono combinate e inserite nella tabella.

Indipendentemente dalla randomizzazione, i campi *prezzo_corrente* e *prezzo_casa_corrente* vengono inizializzati in base ai valori di *costo_acquisto* e *casa*, e moltiplicati in base al *livello_difficolta* della partita. Per le proprietà dello stesso *colore*, viene calcolata la media di *prezzo_casa_corrente*.

La tabella **Giocatore** contiene le informazioni sui partecipanti di ogni partita. Ogni giocatore è associato a:

- Un *tipo* (admin, giocatore o imprenditore)
- Una pedina, garantita unica per partita grazie al vincolo UNIQUE (*idpedina*, *idpartita*)

Durante la partita, la tabella **Partita_Opportunita** registra lo stato delle caselle pescate. Ogni carta è inizialmente marcata come non utilizzata (*utilizzato* = false). Quando una carta viene pescata:

- Il campo *Utilizzato* viene impostato a true
- Se tutte le carte dello stesso tipo sono state utilizzate, vengono resettate a false.
- Se una carta è associata a un giocatore (*idgiocatore* non NULL), non viene mai resettata, poiché posseduta dal giocatore.

4 CAPITOLO: IMPLEMENTAZIONE

4.1 INTRODUZIONE

Il progetto NewMonopoly è stato sviluppato da un team composto da quattro sviluppatori: Messa, Ronchi, Tentori e Teruzzi.

Il lavoro è stato suddiviso secondo le competenze dei membri del team. In particolare, Ronchi e Tentori si sono dedicati prevalentemente allo sviluppo backend, mentre Messa e Teruzzi si sono occupati prevalentemente della realizzazione del front-end.

L'applicazione è stata sviluppata utilizzando il linguaggio di programmazione **Java** per il backend, mentre per il frontend **JavaScript**, **HTML** e **CSS**.

Per quanto riguarda il backend, è stato adottato il framework **Spring Boot**, che ha consentito di costruire un'architettura robusta e scalabile. Sul lato front-end, invece, è stato utilizzato il framework **React**, scelto per la sua flessibilità e per la capacità di creare interfacce utente moderne e interattive.

Per la gestione dei dati, il team ha optato per **PostgreSQL** come sistema di database relazionale, garantendo una gestione sicura ed efficiente delle informazioni. Inoltre, l'infrastruttura del progetto ha fatto uso dei servizi cloud di **AWS** (Amazon Web Service), che ci hanno permesso di implementare un ambiente di sviluppo e distribuzione altamente affidabile e scalabile.

Infine, per le attività di testing e debugging, è stato utilizzato **Ngrok**, uno strumento che ha facilitato la creazione di tunnel sicuri per i test dell'applicazione in ambienti locali, rendendo più agevole il processo di verifica delle funzionalità implementate.

4.2 PATTERN E PRINCIPI SOLID

Nella progettazione del nostro sistema software, abbiamo adottato un insieme di pattern architetturali, design pattern e principi SOLID per garantire modularità, scalabilità e manutenibilità.

4.2.1 Pattern Architetturali

Event-Driven Architecture: L'architettura del sistema si basa su eventi, dove il backend agisce come un dispatcher centralizzato per i messaggi ricevuti tramite socket. Ogni messaggio viene interpretato e smistato in base al tipo specifico, e i componenti del sistema reagiscono agli eventi in modo asincrono e indipendente. Questo approccio garantisce un'elevata modularità e scalabilità, permettendo a diversi componenti di funzionare senza dipendere direttamente l'uno dall'altro. L'adozione di un modello event-driven migliora anche la capacità del sistema di rispondere rapidamente ai cambiamenti o alle richieste in tempo reale.

Service Layer Pattern: Questo pattern prevede la definizione di una classe dedicata per la gestione della logica di business che collega il backend e il frontend. Centralizzando la logica applicativa nel livello di servizio, il sistema risulta più modulare e le interazioni tra i componenti sono chiaramente definite. Inoltre, questa struttura semplifica gli aggiornamenti del sistema, consentendo di modificare o estendere la logica senza impattare direttamente altri livelli dell'architettura. La classe `MessageService`, ad esempio, funge da intermediario per orchestrare la comunicazione tra i vari moduli.

4.2.2 Design Pattern:

Dispatcher Pattern: Questo pattern viene utilizzato per centralizzare e semplificare la logica di smistamento dei messaggi. Il dispatcher analizza i messaggi ricevuti e decide dinamicamente quale classe o modulo debba occuparsi della gestione specifica. Separando la logica di Routing da quella di

elaborazione degli eventi, si riduce significativamente la complessità del sistema, rendendolo più leggibile e mantenibile. Un esempio è rappresentato dalla classe `WebSocketConnectorDispatcher`, che centralizza il flusso dei messaggi.

Repository Pattern: Questo pattern si occupa di astrarre l'accesso al database, separando completamente la logica applicativa dalla persistenza dei dati. Fornisce un'interfaccia chiara e ben definita per l'esecuzione delle operazioni sul database, come il recupero o la manipolazione dei dati. Questa separazione non solo migliora la testabilità del codice, ma consente anche di sostituire o modificare l'implementazione del database senza impattare il resto del sistema. Il risultato è una maggiore flessibilità e facilità di manutenzione.

Manager/Handler Pattern (Component-Based Design): Questo pattern organizza il sistema in componenti con responsabilità ben definite. Manager e Handler sono responsabili della gestione di specifiche aree funzionali, come `TurnManager` per i turni di gioco o `GameHandler` per la gestione delle partite. Questo approccio riduce le dipendenze tra i componenti, facilitando l'aggiunta di nuove funzionalità o l'estensione di quelle esistenti senza influire negativamente sul codice già implementato.

Singleton: Questo pattern centralizza la gestione dello stato di gioco, fornendo un punto unico di accesso e assicurando coerenza tra i vari componenti. Un esempio è la classe `GameBoardSingleton`, utilizzata per garantire una singola istanza condivisa in tutto il sistema. È utile a evitare duplicazioni di dati e semplifica la sincronizzazione, migliorando la robustezza e la manutenibilità del sistema.

Observer: Questo pattern permette di definire una relazione tra un soggetto e i suoi osservatori, in cui il soggetto notifica automaticamente tutti gli osservatori quando il suo stato cambia. Questo pattern è utile per gestire la reattività di un sistema e assicurare che gli aggiornamenti siano propagati ai componenti interessati. Nel nostro progetto, l'Observer è stato applicato attraverso l'uso di `useEffect` di React, che osserva i cambiamenti nelle dipendenze dichiarate e consente ai componenti di reagire automaticamente a tali modifiche. Questo approccio garantisce una gestione reattiva ed efficiente dello stato e del ciclo di vita dei componenti nel front-end.

4.2.3 Principi Solid:

Single Responsibility Principle (SRP): Ogni classe o componente del sistema ha una responsabilità unica e ben definita. Ad esempio, `TurnManager` è incaricato esclusivamente della gestione dei turni di gioco, mentre `GameHandler` si occupa della gestione delle partite. Questo principio migliora la coesione del codice, facilitandone la manutenibilità e la comprensione.

Open/Closed Principle (OCP): La struttura event-driven, insieme all'uso di dispatcher e repository, consente al sistema di essere esteso con nuove funzionalità senza dover modificare il codice esistente. Questo approccio riduce il rischio di introdurre bug durante l'aggiunta di nuove feature, mantenendo il codice stabile e facilmente scalabile.

4.3 REFACTORING

Nel corso del progetto, abbiamo applicato tecniche di refactoring per migliorare la qualità del nostro codice. In particolare, abbiamo identificato alcune criticità utilizzando strumenti come Sonar Cloud e Understand, che ci hanno permesso di rilevare, e per alcuni casi correggere, code smell e antipattern.

Le principali azioni di refactoring effettuate includono:

- **Estrazione di metodi:** abbiamo suddiviso funzioni troppo lunghe e complesse in metodi più piccoli, migliorando la leggibilità e la riusabilità del codice.

- **Rimozione della duplicazione:** abbiamo centralizzato la logica comune in metodi condivisi per ridurre la ripetizione del codice e semplificare la manutenzione.
- **Riorganizzazione delle classi:** abbiamo migliorato la coesione delle classi, riducendo l'accoppiamento e semplificando la loro manutenzione

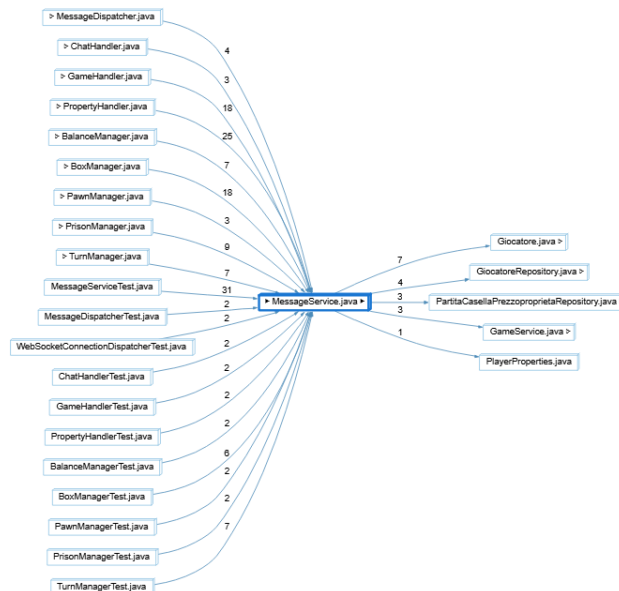
4.4 TEST E QUALITÀ DEL CODICE

Durante il progetto abbiamo implementato test approfonditi per il back-end, ottenendo una copertura del codice pari all'85,5%. I test sono stati eseguiti per garantire la correttezza delle funzionalità implementate.

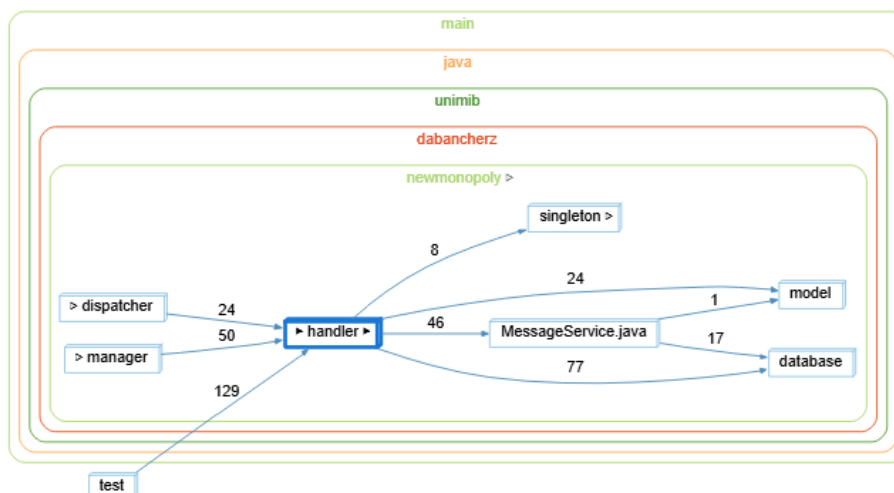
Grazie a Sonar Cloud, il codice ha superato i controlli di sicurezza, affidabilità e manutenibilità con un rating A in tutte le categorie e una percentuale di codice duplicato pari a 0%.

Con l'ausilio di Understand, abbiamo identificato e analizzato possibili antipattern strutturali:

- **External Butterfly:** la classe MessageService ha molte dipendenze esterne al package.



- **Global Butterfly:** il package Database ha numerose dipendenze globali.



- Long Method: presente su SendBoxUsage.
- LargeClass: rilevato su PropertyHandler e PartitaCasellaPrezzoproprietaRepository.
- LongParameterList: identificato in BoxManager e in PlayerProperties.
- GodClass: possibile presenza in MessageDispatcher e SpecialMessageDispatcher.

